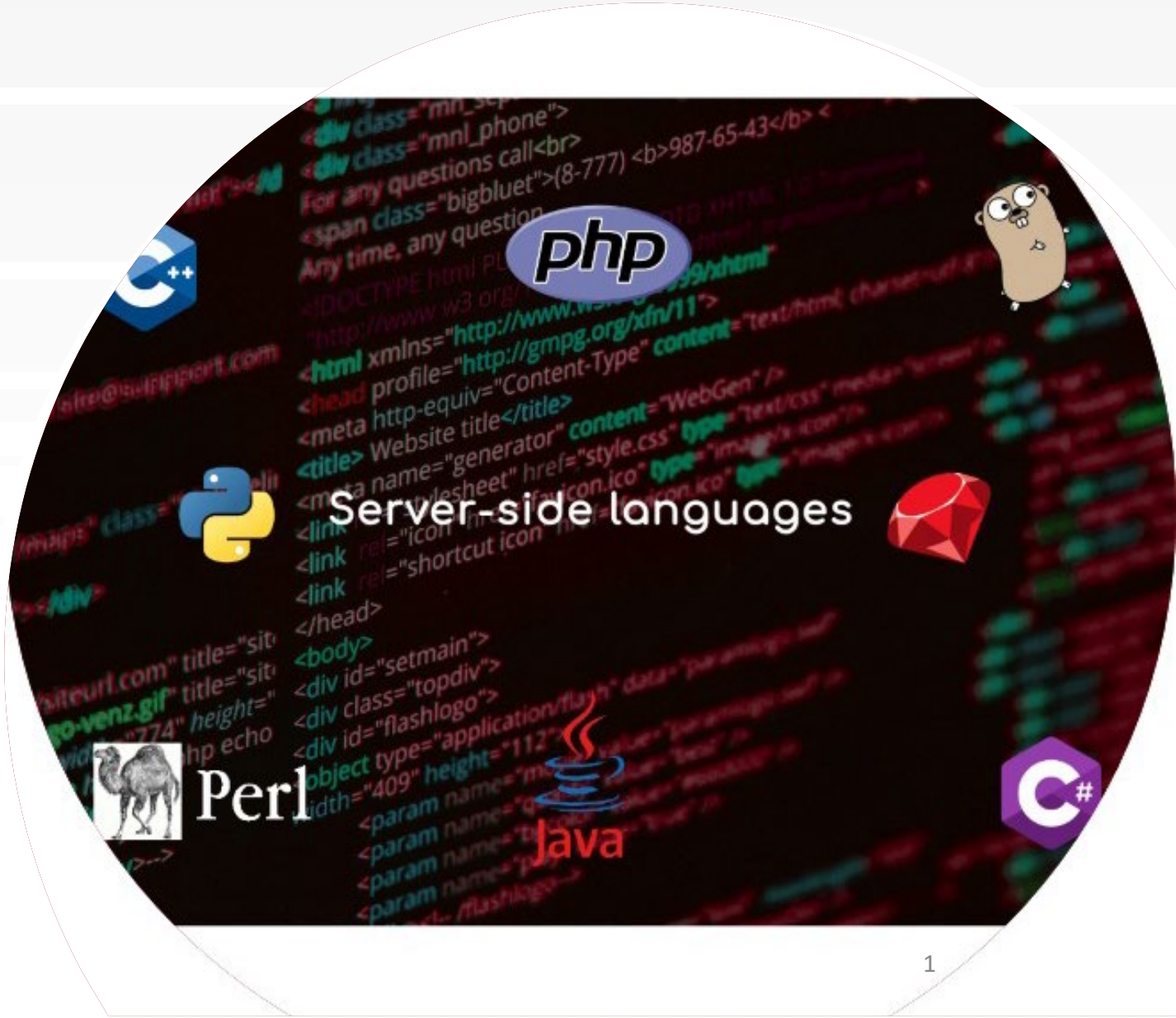
INF1005
Topics for Week 06:

- Form Processing with HML5 and PHP
  - Client-side form validation
  - Server-side form validation and sanitization
  - Security concerns



FORM PROCESSING WITH HTML5 & PHP

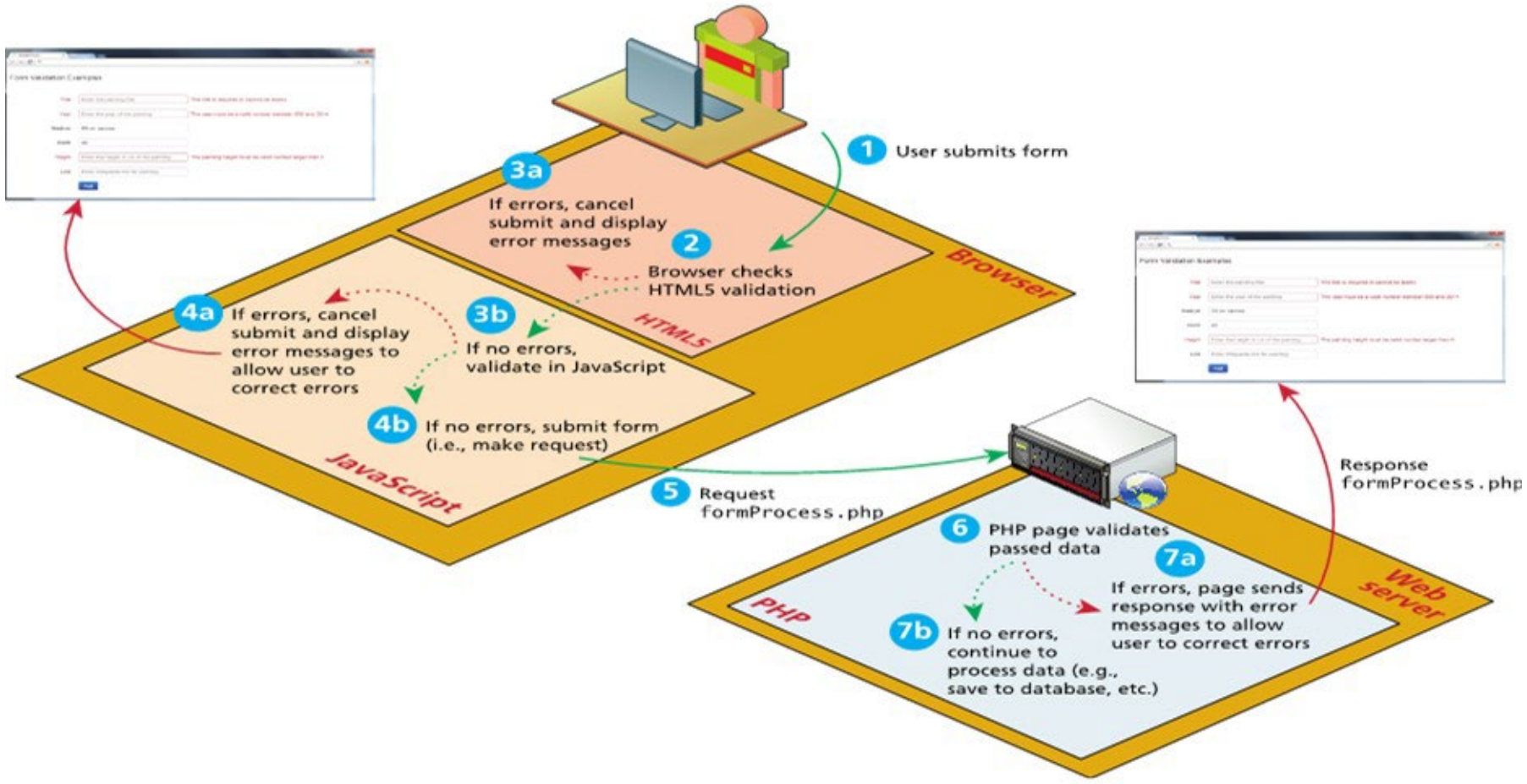
- Client-side Form Processing
  - HTML5 Validation
  - JavaScript Validation
  - Responsive Forms with Bootstrap
- Server-side Form Validation & Sanitization
  - POST vs. GET
  - Security Concerns - XSS & SQL Injection
  - filter_var()
  - htmlspecialchars()
  - trim()
  - stripslashes()
- Why perform BOTH client and server validation?

Where to Perform Validation

Validation can be performed at three different levels.

- With HTML5, the browser can perform **basic validation**.
- **JavaScript validation** dramatically improves the user experience of data-entry forms, and is an essential feature of any real-world web site that uses forms. Unfortunately, JavaScript validation cannot be relied on.
- **server-side validation** is arguably the most important since it is the only validation that is guaranteed to run.

Where to Perform Validation



Additional Materials for Self-study

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 5

HTML 2: Tables and Forms

In this chapter you will learn . . .

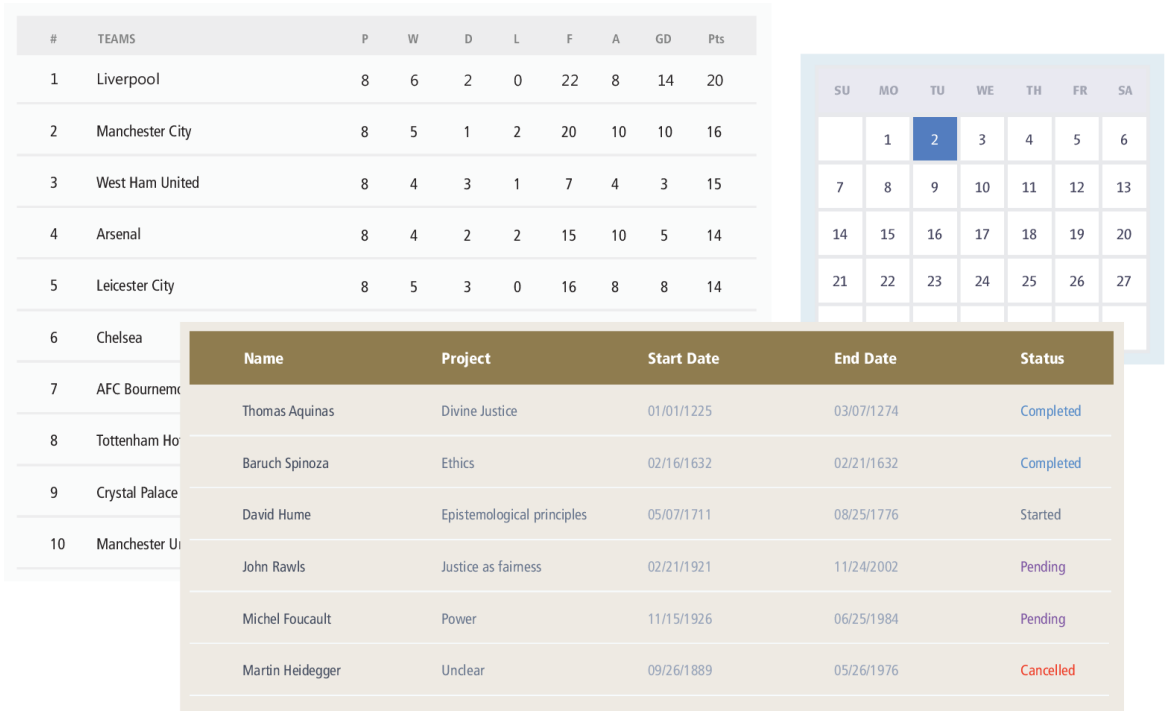
- What HTML tables are and how to create them
- How to use CSS to style tables
- What forms are and how they work
- What the different form controls are and how to use them
- How to improve the accessibility of your websites

HTML Tables

A **table** in HTML is created using the <table> element and can be used to represent information that exists in a two-dimensional grid.

Just like a real-world table, an HTML table can contain any type of data: not just numbers, but text, images, forms, even other tables

<table> contains any number of rows (<tr>); each row contains any number of table data cells (<td>)



#	TEAMS	P	W	D	L	F	A	GD	Pts
1	Liverpool	8	6	2	0	22	8	14	20
2	Manchester City	8	5	1	2	20	10	10	16
3	West Ham United	8	4	3	1	7	4	3	15
4	Arsenal	8	4	2	2	15	10	5	14
5	Leicester City	8	5	3	0	16	8	8	14
6	Chelsea								
7	AFC Bourneme								
8	Tottenham Ho								
9	Crystal Palace								
10	Manchester U								

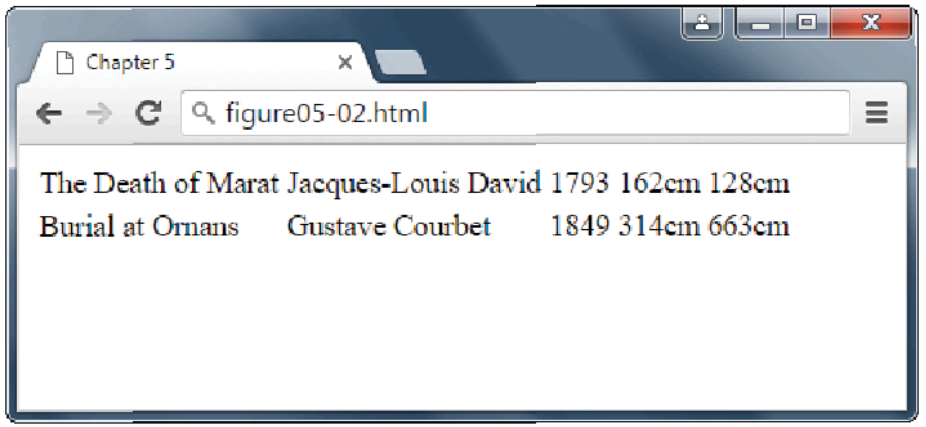
SU	MO	TU	WE	TH	FR	SA
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27

Name	Project	Start Date	End Date	Status
Thomas Aquinas	Divine Justice	01/01/1225	03/07/1274	Completed
Baruch Spinoza	Ethics	02/16/1632	02/21/1632	Completed
David Hume	Epistemological principles	05/07/1711	08/25/1776	Started
John Rawls	Justice as fairness	02/21/1921	11/24/2002	Pending
Michel Foucault	Power	11/15/1926	06/25/1984	Pending
Martin Heidegger	Unclear	09/26/1889	05/26/1976	Cancelled

Basic table structure

<table>
<tr>
<td>The Death of Marat</td>
<td>Jacques-Louis David</td>
<td>1793</td>
<td>162cm</td>
<td>128cm</td>
</tr>
<tr>
<td>Burial at Ornans</td>
<td>Gustave Courbet</td>
<td>1849</td>
<td>314cm</td>
<td>663cm</td>
</tr>
</table>

```
<table>
   <tr>
      <td>The Death of Marat</td>
      <td>Jacques-Louis David</td>
      <td>1793</td>
      <td>162cm</td>
      <td>128cm</td>
   </tr>
   <tr>
      <td>Burial at Ornans</td>
      <td>Gustave Courbet</td>
      <td>1849</td>
      <td>314cm</td>
      <td>663cm</td>
   </tr>
</table>
```



Spanning Rows and Columns

- All content must appear within the <td> or <th> container.
- Each row must have the same number of <td> or <th> containers.
- If you want a given cell to cover several columns or rows, then you can do so by using the **colspan** or **rowspan** attributes

FIGURE 5.4 Spanning columns

<table>

<tr>	Title <th>	Artist <th>	Year <th>	Size (width x height) <th colspan=2>	
<tr>	The Death of Marat <td>	Jacques-Louis David <td>	1793 <td>	162cm <td>	128cm <td>
<tr>	Burial at Ornans <td>	Gustave Courbet <td>	1849 <td>	314cm <td>	663cm <td>

Notice that this row now only has four cell elements.

```
<table>
   <tr>
      <th>Title</th>
      <th>Artist</th>
      <th>Year</th>
      <th colspan="2">Size (width x height)</th>
   </tr>
   <tr>
      <td>The Death of Marat</td>
      <td>Jacques-Louis David</td>
      <td>1793</td>
      <td>162cm</td>
      <td>128cm</td>
   </tr>
   ...
</table>
```

Additional table elements

A title for the table is good for accessibility.

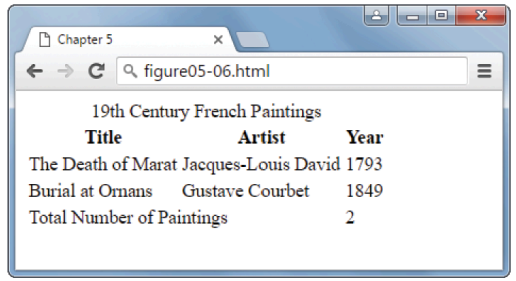
These describe our columns and can be used to aid in styling.

Table header could potentially also include other <tr> elements.

Yes, the table footer comes *before* the body.

Potentially, with styling, the browser can scroll this information while keeping the header and footer fixed in place.

```
<table>
   <caption>19th Century French Paintings</caption>
   <col class="artistName" />
   <colgroup id="paintingColumns">
      <col />
      <col />
   </colgroup>

   <thead>
      <tr>
         <th>Title</th>
         <th>Artist</th>
         <th>Year</th>
      </tr>
   </thead>

   <tfoot>
      <tr>
         <td colspan="2">Total Number of Paintings</td>
         <td>2</td>
      </tr>
   </tfoot>

   <tbody>
      <tr>
         <td>The Death of Marat</td>
         <td>Jacques-Louis David</td>
         <td>1793</td>
      </tr>
      <tr>
         <td>Burial at Ornans</td>
         <td>Gustave Courbet</td>
         <td>1849</td>
      </tr>
   </tbody>

</table>
```



Using Tables for Layout

Prior to the broad support for CSS in browsers, HTML tables were frequently used to create page layouts. Unfortunately, this practice of using tables for layout had some problems:

1. This approach tended to increase the size of the HTML document
2. The resulting markup is not semantic. Using <table> simply to get two block elements side by side is an example of using markup simply for presentation reasons
3. Using tables for layout results in a page that is not accessible

The next chapter will examine how to use CSS for layout purposes.

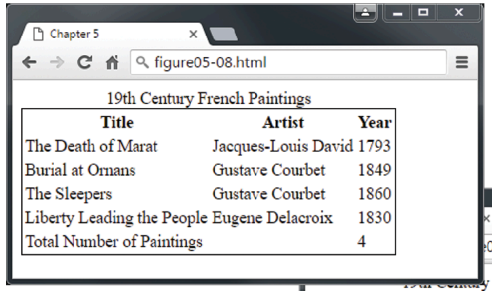
Styling Tables (Borders)

Borders can be assigned to both the **<table>** and the **<td>** element. Interestingly, borders cannot be assigned to the <tr>, <thead>, <tfoot>, and <tbody> elements.

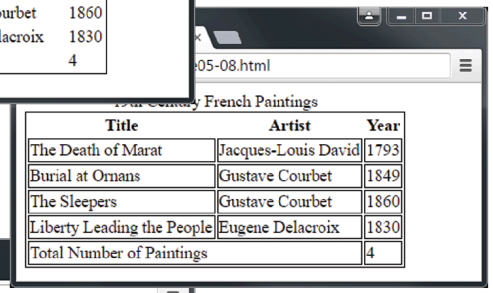
Notice as well the **border-collapse** property. This property selects the table's border model.

- The default is the **separated** model or value (each cell has its own unique borders)
- The **collapsed** border model makes adjacent cells share a single border.

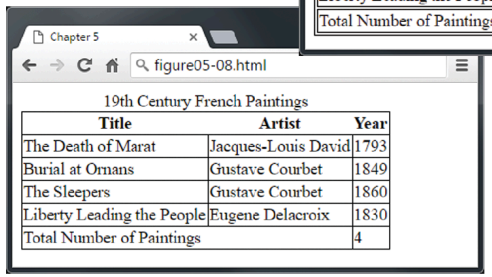
Styling table borders



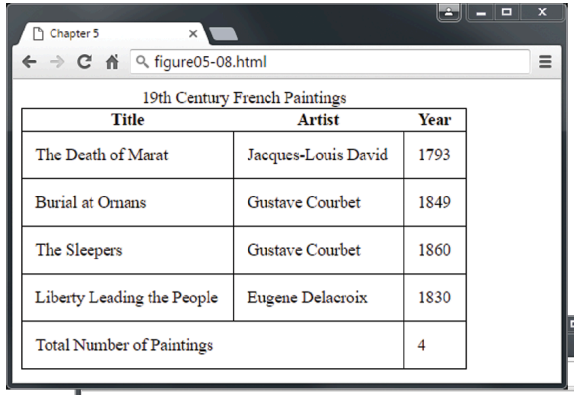

```
table {
   border: solid 1pt black;
}
```



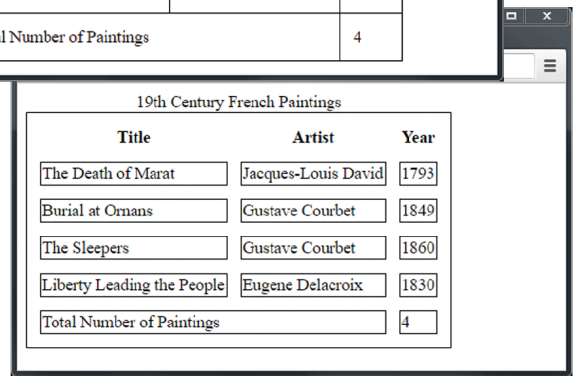

```
table {
   border: solid 1pt black;
}
td {
   border: solid 1pt black;
}
```




```
table {
   border: solid 1pt black;
   border-collapse: collapse;
}
td {
   border: solid 1pt black;
}
```



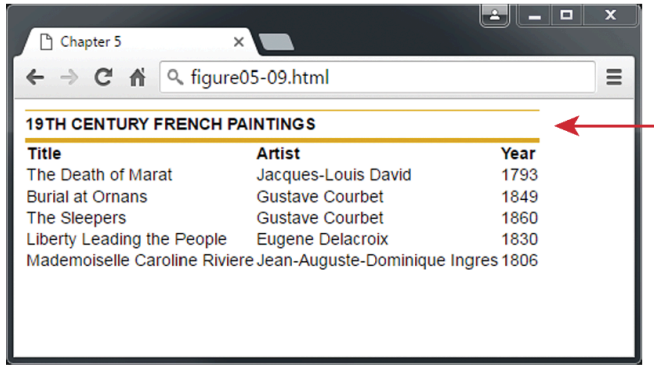

```
table {
   border: solid 1pt black;
   border-collapse: collapse;
}
td {
   border: solid 1pt black;
   padding: 10pt;
}
```




```
table {
   border: solid 1pt black;
   border-spacing: 10pt;
}
td {
   border: solid 1pt black;
}
```

Boxes and Zebras

- While there is almost no end to the different ways one can style a table, there are a number of common approaches. We will look at two of them here.
- The first of these is a box format, in which we simply apply background colors and borders in various ways

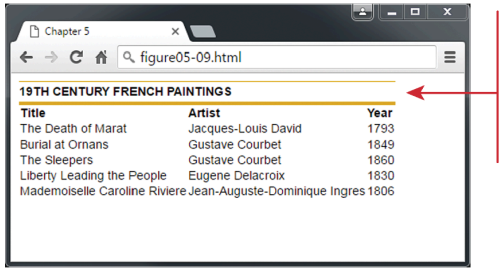



```
caption {
   font-weight: bold;
   padding: 0.25em 0 0.25em 0;
   text-align: left;
   text-transform: uppercase;
   border-top: 1px solid #DCA806;
}
table {
   font-size: 0.8em;
   font-family: Arial, sans-serif;
   border-collapse: collapse;
   border-top: 4px solid #DCA806;
   border-bottom: 1px solid white;
   text-align: left;
}
```

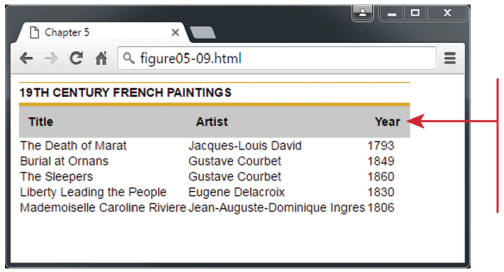
Boxes and Zebras (ii)

While there is almost no end to the different ways one can style a table, there are a number of common approaches. We will look at two of them here.

- The first of these is a box format, in which we simply apply background colors and borders in various ways
- See how the pseudo-element nth-child (covered in Chapter 4) can be used to alternate the format of every second row




```
caption {
    font-weight: bold;
    padding: 0.25em 0 0.25em 0;
    text-align: left;
    text-transform: uppercase;
    border-top: 1px solid #DCA806;
}
table {
    font-size: 0.8em;
    font-family: Arial, sans-serif;
    border-collapse: collapse;
    border-top: 4px solid #DCA806;
    border-bottom: 1px solid white;
    text-align: left;
}
```




```
thead tr {
    background-color: #CACACA;
}
th {
    padding: 0.75em;
}
```

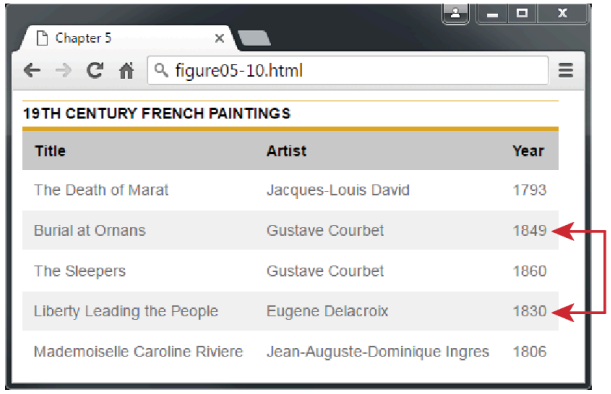

```
tbody tr {
    background-color: #F1F1F1;
    border-bottom: 1px solid white;
    color: #6E6E6E;
}
tbody td {
    padding: 0.75em;
}
```

Boxes and Zebras (iii)

We can then

- add special styling to the :hover pseudo-class of the <tr> element to highlight a row when the mouse cursor hovers over
- use the pseudo-element nth-child (covered in Chapter 4) to alternate the format of every second row.


```
tbody tr:hover {
   background-color: #9e9e9e;
   color: black;
}
```



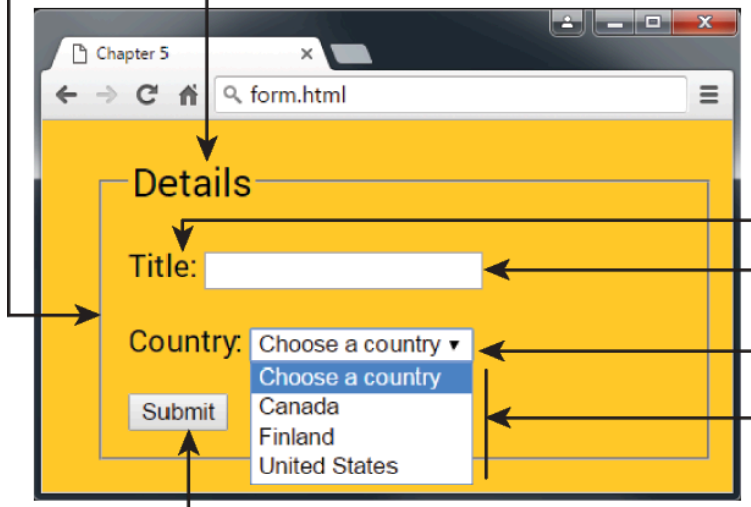

```
tbody tr:nth-child(even) {
   background-color: lightgray;
}
```

Introducing Forms

Forms provide the user with an alternative way to interact with a web server.

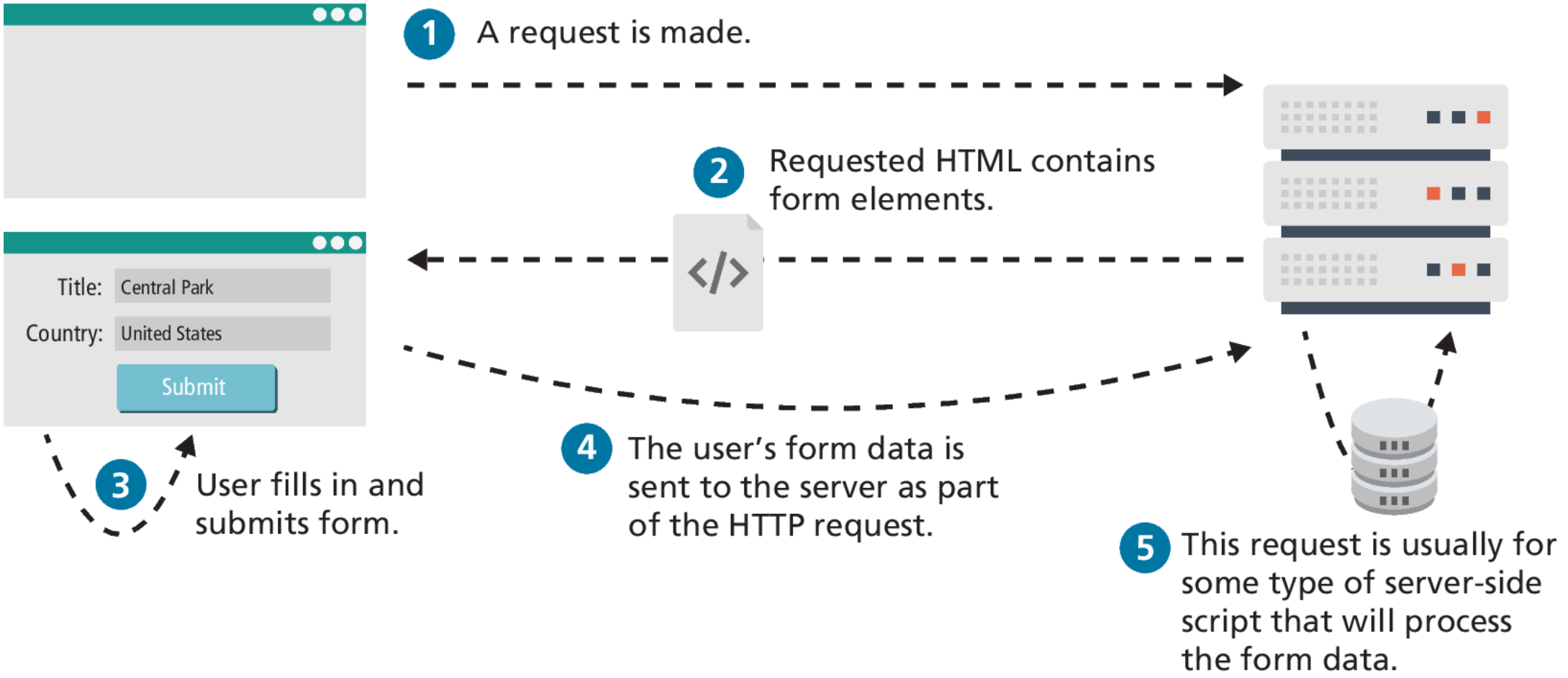
- Up to now, clicking hyperlinks was the only mechanism available to the user
- Using a form, the user can enter text, choose items from lists, and click buttons
- Typically, programs running on the server will take the input from HTML forms and do something with it,
  - such as save it in a database,
  - interact with an external web service, or
  - customize subsequent HTML based on that input.
- HTML5 has added a number of new controls and more customization options

Form Structure




```
<form method="post" action="process.php">
 <fieldset>
  <legend>Details</legend>
  <p>
   <label>Title: </label>
   <input type="text" name="title" />
  </p>
  <p>
    <label>Country: </label>
   <select name="where">
    <option>Choose a country</option>
    <option>Canada</option>
    <option>Finland</option>
    <option>United States</option>
   </select>
  </p>
  <input type="submit" />
 </fieldset>
</form>
```

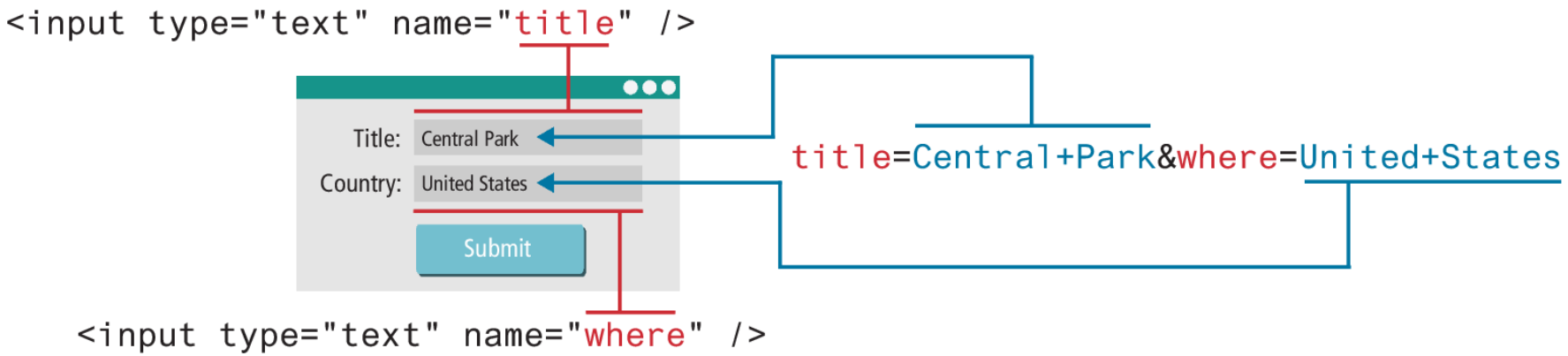
How Forms Work



Query Strings

The browser “sends” the data to the server via an HTTP request using a query string.

A **query string** is a series of **name=value** pairs separated by ampersands (the & character). Special symbols must be **URL encoded**



The <form> Element

Two important attributes that are essential features of any <form> are the **action** and the **method** attributes.

- The **action** attribute specifies the URL of the server-side resource that will process the form data.
- The **method** attribute specifies how the query string data will be transmitted from the browser to the server. There are two possibilities:
  - GET and
  - POST.

GET

GET

- Data can be clearly seen in the address bar. This may be an advantage during development but a disadvantage in production.
- Data remains in browser history and cache. Again this may be beneficial to some users, but it is a security risk on public computers.
- Data can be bookmarked (also an advantage and a disadvantage).
- There is a limit on the number of characters in the returned form data.

POST

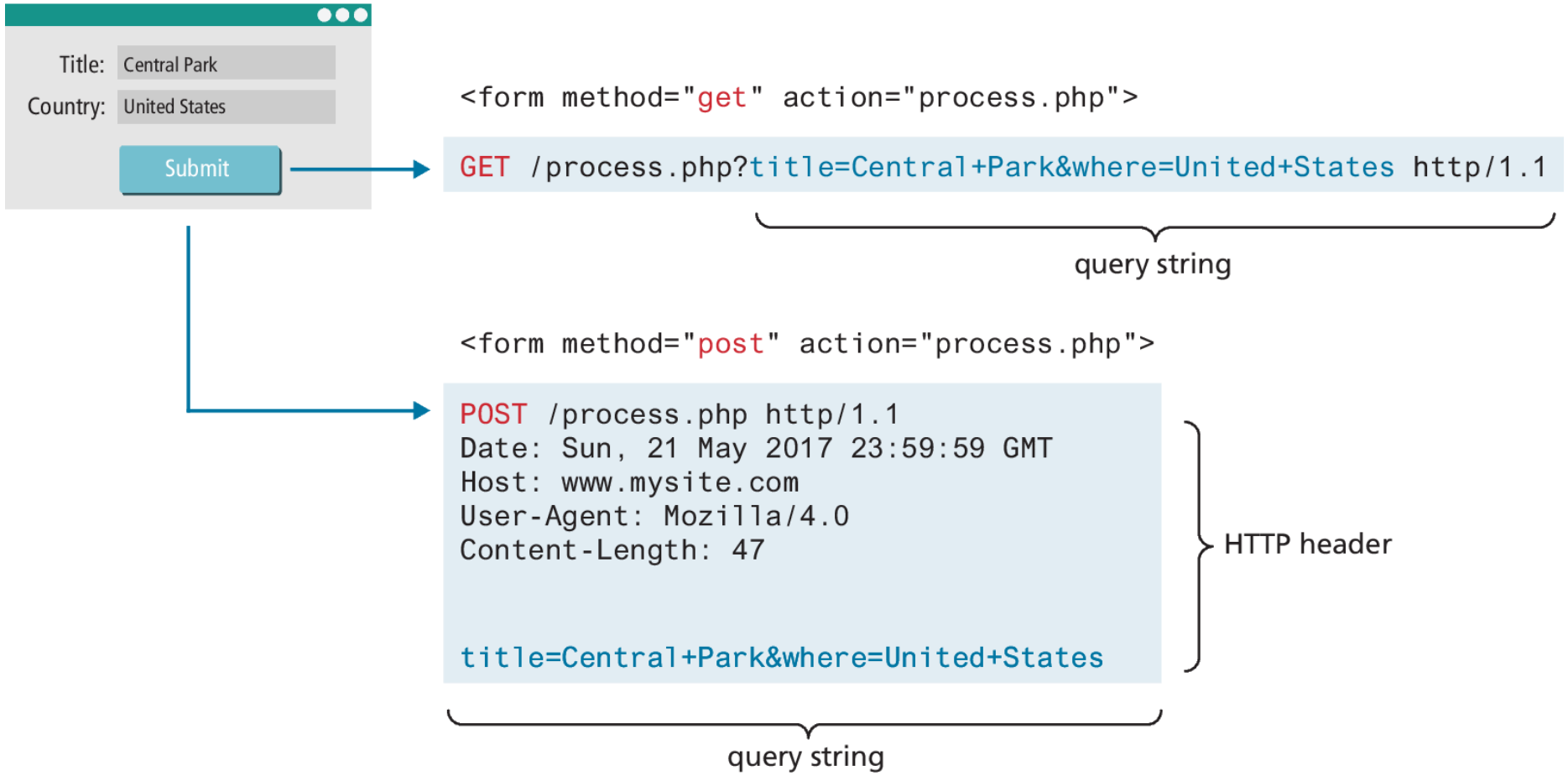
POST

- Data can contain binary data.
- Data is hidden from user.
- Submitted data is not stored in cache, history, or bookmarks.

NOTE

It should be noted that while the POST method “hides” form data, any user could easily inspect the HTTP header. As a result, the POST method is NOT sufficient from a security standpoint.

GET vs POST (example)



Form Control Elements

<button> Defines a clickable button.

<datalist> An HTML5 element that defines lists of pre-defined values to use with input fields.

<fieldset> Groups related elements in a form together.

<form> Defines the form container.

<input> Defines an input field. HTML5 defines over 20 different types of input.

<label> Defines a label for a form input element.

<legend> Defines the label for a fieldset group.

<optgroup> Defines a group of related options in a multi-item list.

<option> Defines an option in a multi-item list.

<output> Defines the result of a calculation.

<select> Defines a multi-item list.

<textarea> Defines a multiline text entry box.

Text Input Controls

Most forms need to gather text information from the user. Whether it is a search box or a login form or a user registration form, some type of text input is usually necessary.

```
<input type="text" ... />
```

Text:

```
<textarea>
  enter some text
</textarea>
```

```
<textarea placeholder="enter some text">
</textarea>
```

TextArea: enter some text

TextArea: Enter some text

```
<input type="password" ... />
```

Password:

Password: ••••

```
<input type="search" placeholder="enter search text" ... />
```

Search: enter search text

Search: HTML

```
<input type="email" ... />
```

Email: fdsdfs

Please enter a valid email address

In Opera

Email: sdasdas

Please enter an email address.

In Chrome

```
<input type="url" ... />
```

url: sdsdfdf

Please enter a URL.

```
<input type="tel" ... />
```

Tel:

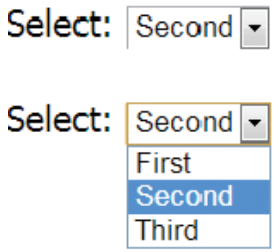
Choice Controls

Forms often need the user to select an option from a group of choices. HTML provides several ways to do this.

- Select Lists
- Radio Buttons
- Checkboxes

Select Lists

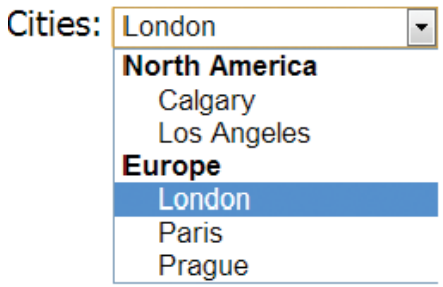
- The **<select>** element is used to create a multiline box for selecting one or more items. The options (defined using the **<option>** element) can be hidden in a dropdown list or multiple rows of the list can be visible.
- The selected attribute in the **<option>** makes it a default value.
- The value attribute is optional; if it is not specified, then the text within the container is sent instead




```
<select name="choices">
  <option>First</option>
  <option selected>Second</option>
  <option>Third</option>
</select>
```

Select Lists (ii)

- Option items can be grouped together via the **<optgroup>** element.




```
<select ... >
 <optgroup label="North America">
   <option>Calgary</option>
   <option>Los Angeles</option>
 </optgroup>
 <optgroup label="Europe">
   <option>London</option>
   <option>Paris</option>
   <option>Prague</option>
 </optgroup>
</select>
```

Radio Buttons

Radio buttons are useful when you want the user to select a single item from a small list of choices and you want all the choices to be visible.

radio buttons are added via the **<input type="radio">** element

The checked attribute is used to indicate the default choice, while the value attribute works in the same manner as with the **<option>** element.


```
<input type="radio" name="where" value="1">North America<br/>
<input type="radio" name="where" value="2" checked>South America<br/>
<input type="radio" name="where" value="3">Asia
```

Checkboxes

A **checkbox** is used for obtaining a yes/no or on/off response from the user.

Checkboxes are added via the **<input type="checkbox">**

The **checked** attribute can be used to set the default value of a checkbox.

Each checked checkbox will have its value sent to the server.

Button Controls

<input type="submit"> Creates a button that submits the form data to the server.

<input type="reset"> Creates a button that clears any of the user's already entered form data.

<input type="button"> Creates a custom button. This button may require JavaScript for it to actually perform any action.

<input type="image"> Creates a custom submit button that uses an image for its display.

<button> Creates a custom button.

Example button elements

```
<input type="submit"  />
```


```
<input type="reset"  />
```

```
<input type="button" value="Click Me" />
```


```
<input type="image" src="appointment.png" />
```


```
<button>
   <a href="email.html">
      <img src="images/email.png" alt="..."/>
      Email
   </a>
</button>
```

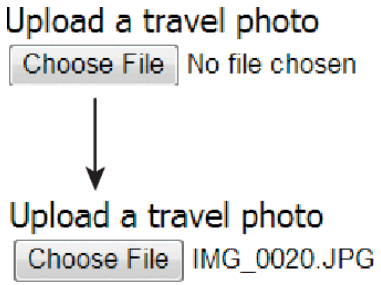
```
<button type="submit"  >
   <img src="images/edit.png" alt="..."/>
   Edit
</button>
```

Specialized Controls

<input type="hidden"> element will be covered in more detail in Chapter 15 on State Management

<input type="file"> element, which is used to upload a file from the client to the server

Notice that the **<form>** element must use the **post** method and must include the **enctype="multipart/form-data"** attribute as well.




```
<form method="post" enctype="multipart/form-data" ... >
    ...
    <label>Upload a travel photo</label>
    <input type="file" name="photo" />
    ...
</form>
```

Number and Range

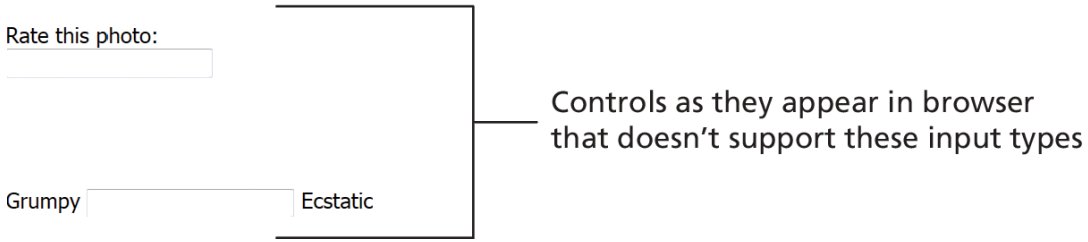
- HTML5 introduced the number and range controls provide a way to input numeric values that eliminates the need for client-side numeric validation (for security reasons you would still check the numbers for validity on the server)

Rate this photo:
2

```
<label>Rate this photo: <br/>
<input type="number" min="1" max="5" name="rate" />
```

Grumpy Ecstatic

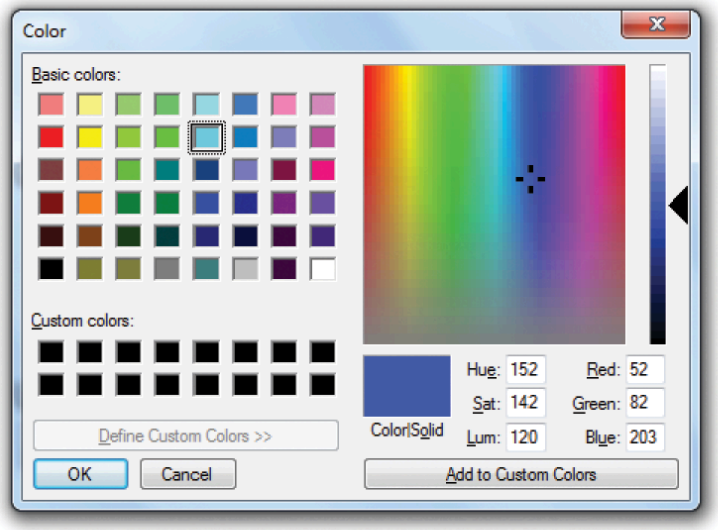
```
Grumpy
<input type="range" min="0" max="10" step="1" name="happiness" />
Ecstatic
```



Color

When it is necessary, the HTML5 color control provides a convenient interface for the user

Background Color:




```
<label>Background Color: <br/>
<input type="color" name="back" />
```

Background Color:

Control as it appears in browser that doesn't support this input type

Date and Time Controls

date Creates a general date input control. The format for the date is “yyyy-mm-dd.”

time Creates a time input control. The format for the time is “HH:MM:SS,” for hours:minutes:seconds.

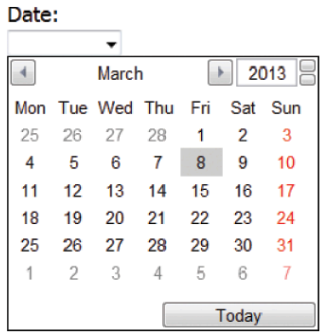
datetime Creates a control in which the user can enter a date and time.

datetime-local Creates a control in which the user can enter a date and time without specifying a time zone.

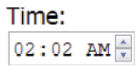
month Creates a control in which the user can enter a month in a year. The format is “yyyy-mm.”

week Creates a control in which the user can specify a week in a year. The format is “yyyy-W##.”

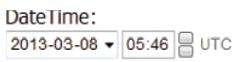
Date and time controls Figure



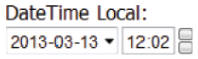

```
<label>Date: <br/>
<input type="date" ... />
```



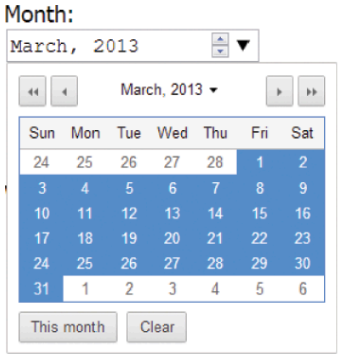

```
<input type="time" ... />
```



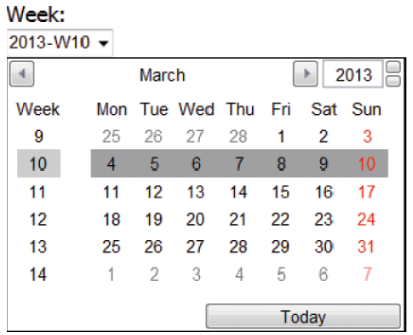

```
<input type="datetime" ... />
```




```
<input type="datetime-local" ... />
```




```
<input type="month" ... />
```




```
<input type="week" ... />
```

Table and Form Accessibility

The W3C created the **Web Accessibility Initiative (WAI)** in 1997 to improve the accessibility of websites. Perhaps the most important guidelines in that document are:

- *Provide text alternatives for any nontext content so that it can be changed into other forms people need, such as large print, braille, speech, symbols, or simpler language*.
- *Create content that can be presented in different ways (for example, simpler layout) without losing information or structure*.
- *Make all functionality available from a keyboard*.
- *Provide ways to help users navigate, find content, and determine where they are*.

Accessible Tables

- Describe the table's content using the <caption> element
- Connect the cells with a textual description in the header using use the **scope** attribute.

```
<table>
  <caption>Famous Paintings</caption>
  <tr>
    <th scope="col">Title</th>
    <th scope="col">Artist</th>
    <th scope="col">Year</th>
    <th scope="col">Width</th>
    <th scope="col">Height</th>
  </tr>
  <tr>
    <td>The Death of Marat</td>
    <td>Jacques-Louis David</td>
    <td>1793</td>
    <td>162cm</td>
    <td>128cm</td>
  </tr>
…
```

LISTING 5.1 (edited) Connecting cells with headers

Accessible Forms

We already made use of the <fieldset>, <legend>, and <label> elements.

Their main purpose is to logically group related form input elements together with the <legend> providing a type of caption for those elements.

Each <label> element should be associated with a single input element.

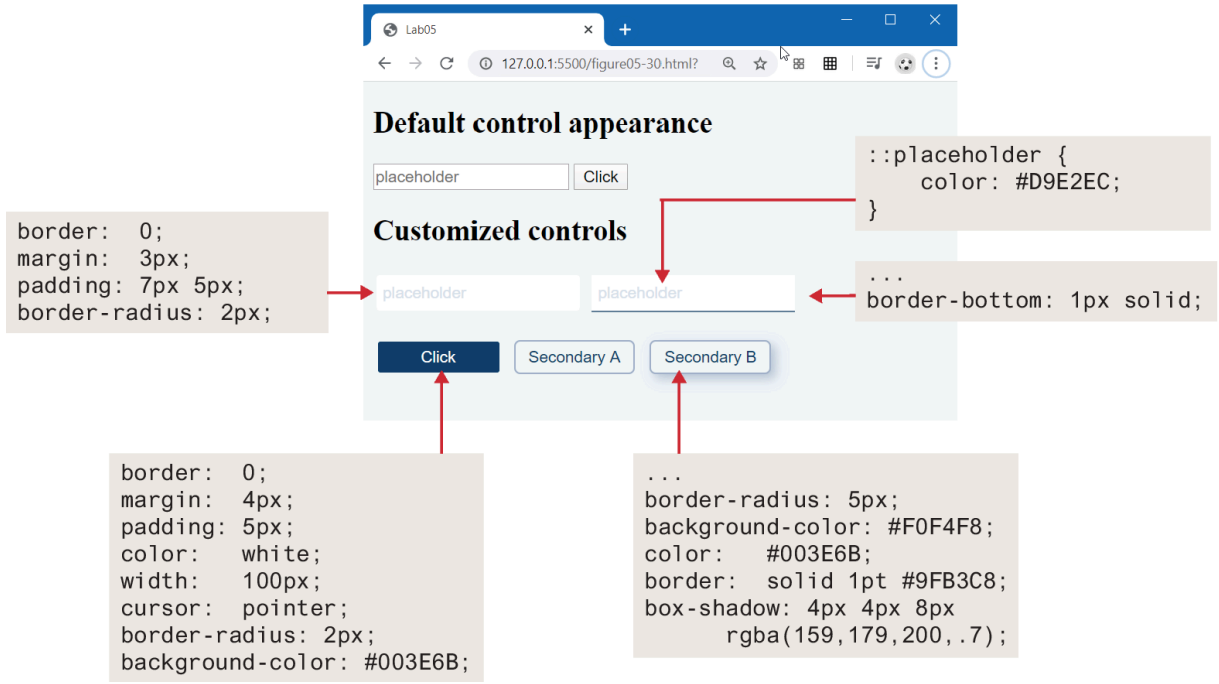
You can make this association explicit by using the for attribute so means that if the user clicks on or taps the <label> text the control will receive the focus

Associating labels and input elements

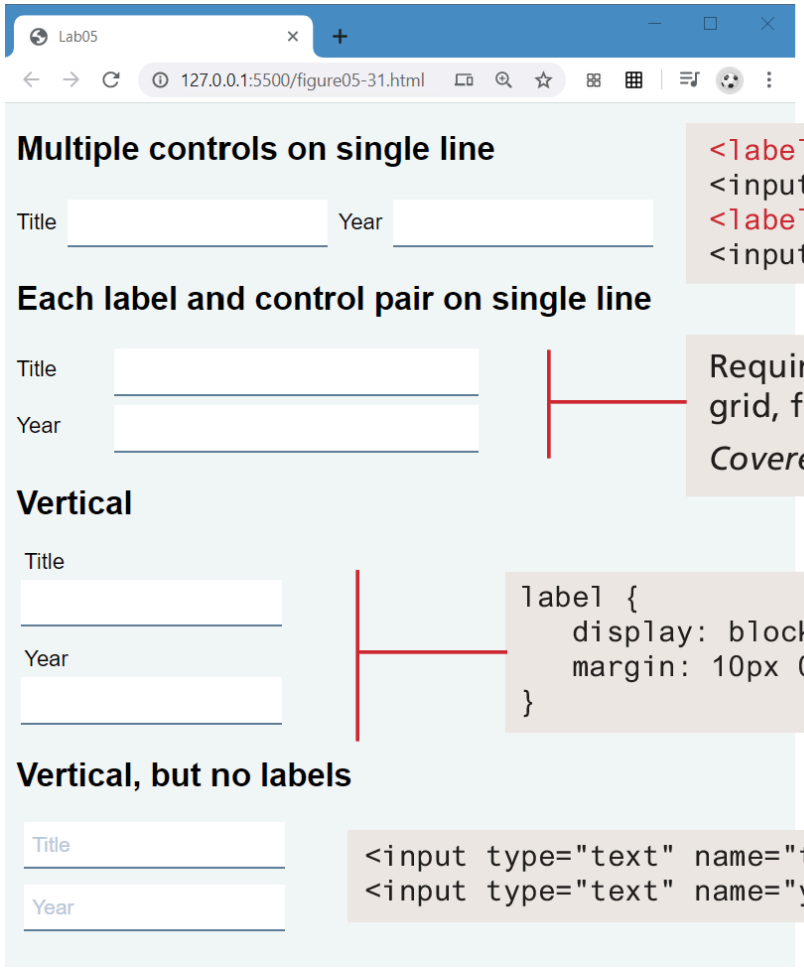
```
<label for="f-title">Title: </label>

<input type="text" name="title" id="f-title"/>


<label for="f-country">Country: </label>

<select name="where" id="f-country">
    <option>Choose a country</option>
    <option>Canada</option>
    <option>Finland</option>
    <option>United States</option>
</select>
```

Styling Form Elements

Let's begin with the common text and button controls. A common styling change is to eliminate the borders and add in rounded corners and padding.



Working with labels




```
<label for="title">Title</label>
<input type="text" name="title" id="title"/>
<label for="year">Year</label>
<input type="text" name="year" id="year" />
```

Requires CSS layout techniques, such as grid, flex, floats, or positioning.

Covered in Chapter 7.

```
label {
   display: block;
   margin: 10px 0 0 5px;
}
```

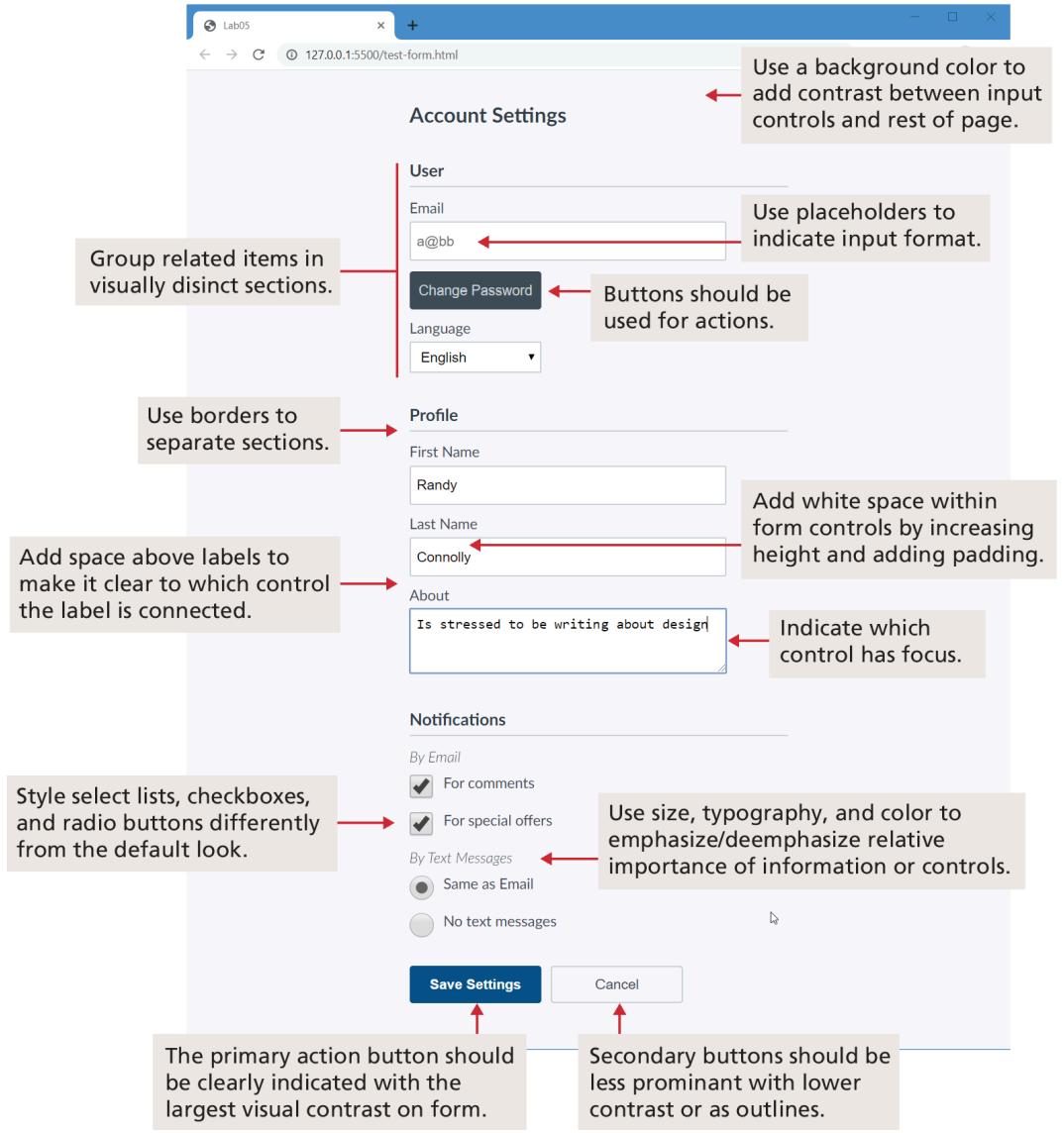
```
<input type="text" name="title" placeholder="Title" />
<input type="text" name="year" placeholder="Year" />
```

Form Design

A well-designed form communicates to a user that the site values their time and data.

Perhaps the first and most important rule is to style your form elements so they look different from the default settings.

Figure 5.33 describes and illustrates a small set of straightforward additional precepts



Validating User Input

- User input must never be trusted.
- It could be missing.
- It might be in the wrong format.
- It might even contain JavaScript or SQL as a means to causing some type of havoc.
- Thus, almost always user input must be tested for validity.

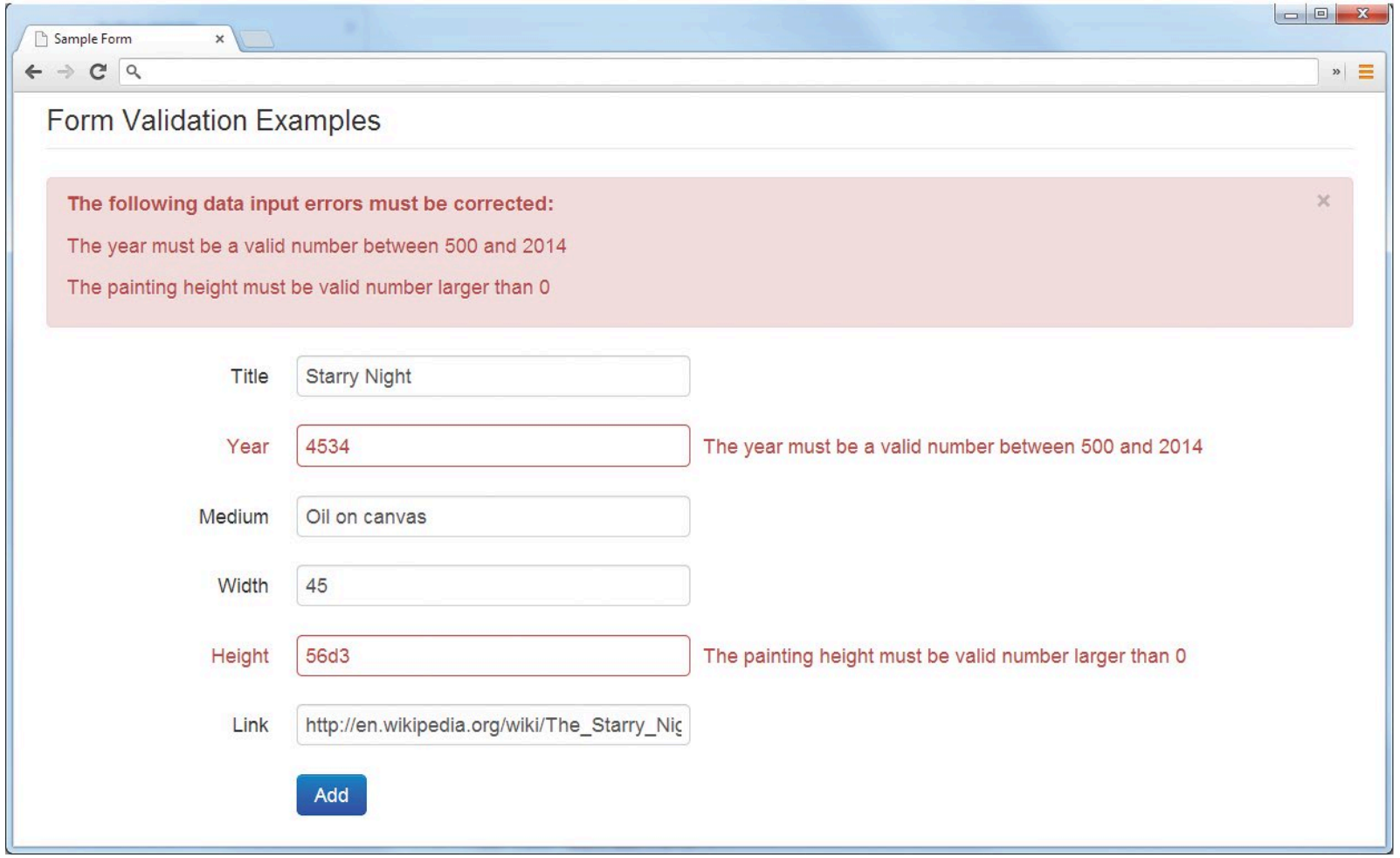
Types of Input Validation

- **Required information.** Some data fields just cannot be left empty.
- **Correct data type.** Some fields such as numbers or dates, must follow the rules for its data type in order to be considered valid.
- **Correct format.** Some information, such as postal codes, credit card numbers, and social security numbers have to follow certain pattern rules.
- **Comparison.** Some user-entered fields are considered correct or not in relation to an already inputted value (password confirm)
- **Range check.**
- **Custom.**

Notifying the User

- **What is the problem?** Users do not want to read lengthy messages to determine what needs to be changed.
- **Where is the problem?** Some type of error indication should be located near the field that generated the problem.
- **If appropriate, how do I fix it?** For instance, don't just tell the user that a date is in the wrong format; tell him or her what format you are expecting

Notifying the User (Figures)



How to Reduce Validation Errors

Provide textual hints to the user on the form itself (last slide)

Using tool tips or pop-overs to display context-sensitive help about the expected input

Another technique for helping the user understand the correct format for an input field is to provide a JavaScript-based mask

Providing sensible default values for text fields can reduce validation errors

Finally, many user input errors can be eliminated by choosing a better data entry type than the standard <input type="text">.

Where to Perform Validation

Validation can be performed at three different levels.

- With HTML5, the browser can perform **basic validation**.
- **JavaScript validation** dramatically improves the user experience of data-entry forms, and is an essential feature of any real-world web site that uses forms. Unfortunately, JavaScript validation cannot be relied on.
- **server-side validation** is arguably the most important since it is the only validation that is guaranteed to run.

Key Terms

- accessibility
- branch
- CAPTCHA
- checkbox
- forking
- form
- GET
- Git
- GitHub
- input validation
- local repository
- POST
- query string
- radio buttons
- remote repository
- spam bots
- table
- URL encoded
- version control
- Web Accessibility Initiative (WAI)

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 16

Security

In this chapter you will learn . . .

- About core security principles and practices
- Best practices for authentication systems and data storage
- About public key cryptography, SSL, and certificates
- How to proactively protect your site against common attacks

Security Principles

Security theory and practice will guide you in that never-ending quest to defend your data and systems, which you will see, touches all aspects of software development.

Not only is a malicious hacker on a tropical island a threat but so too is a sloppy programmer, a disgruntled manager, or a naive secretary. Moreover, threats are ever changing, and with each new counter measure, new threats emerge to supplant the old ones.

You must draw from those fields to learn many foundational security ideas. Later, you will apply these ideas to harden your system against malicious users and defend against programming errors.

Information Security

Information security is the holistic practice of protecting information from unauthorized users.

Information assurance ensures that data is not lost when issues do arise.

At the core of information security is the **CIA triad**: *confidentiality*, *integrity,* and *availability*

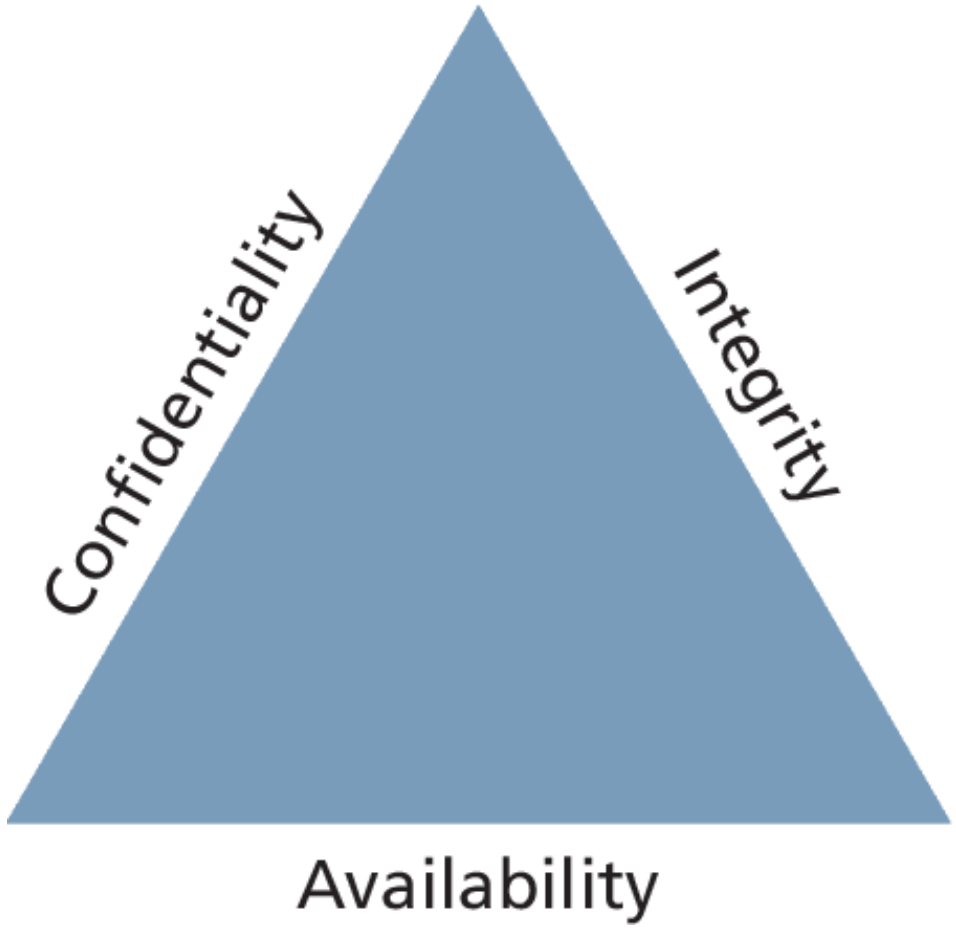
In addition to the triad, there are ISO standards ISO/IEC 27002-270037 that speak directly (and thoroughly) about security techniques and are routinely adopted by governments and corporations the world over.

CIA Triad

Confidentiality is the principle of maintaining privacy for the data you are storing, transmitting, and so forth

Integrity is the principle of ensuring that data is accurate and correct.

Availability is the principle of making information available to authorized people when needed.



Risk Assessment and Management

Risk assessment uses the concepts of actors, impacts, threats, and vulnerabilities to determine where to invest in defensive countermeasures.

- **Actors** refers to the people who are attempting to access your system. They can be categorized as internal, external, and partners.
- The **impact** of an attack depends on what systems were infiltrated and what data was stolen or lost. The impact relates back to the CIA triad since impact could be the loss of availability, confidentiality, and/or integrity
- A **threat** refers to a particular path that a hacker could use to exploit a vulnerability and gain unauthorized access to your system.
- **Vulnerabilities** are the security holes in your system.

STRIDE

Broadly, threats can be categorized using the **STRIDE** mnemonic, developed by Microsoft, which describes six areas of threat:

- **S**poofing—The attacker uses someone else's information to access the system.
- **T**ampering—The attacker modifies some data in nonauthorized ways.
- **R**epudiation—The attacker removes all trace of their attack so that they cannot be held accountable for other damages done.
- **I**nformation disclosure—The attacker accesses data they should not be able to.
- **D**enial of service—The attacker prevents real users from accessing the systems.
- **E**levation of privilege—The attacker increases their privileges on the system, thereby getting access to things they are not authorized to.

Assessing Risk

In risk assessment, you begin by identifying the **actors**, **vulnerabilities**, and **threats** to your information systems.

Table 16.1 illustrates the relationship between the probability of an attack and its impact on an organization.

The table weighs impact on the *x* scale and probability on the *y* scale.

Probability \ Impact (n^2)	Very low	Low	Medium	High	Very high
Very high	5	10	20	40	80
High	4	8	16	32	64
Medium	3	6	12	24	48
Low	2	4	8	16	32
Very low	1	2	4	8	16

Security Policy

- **Usage policy** defines what systems users are permitted to use, and under what situations.
- **Authentication policy** controls how users are granted access to the systems. These policies most often manifest as simple **password policies**
- **Legal policies** define a wide range of things including data retention and backup policies as well as accessibility requirements

Good policies aim to modify the behavior of internal actors, but will not stop foolish or malicious behavior by employees.

Business Continuity

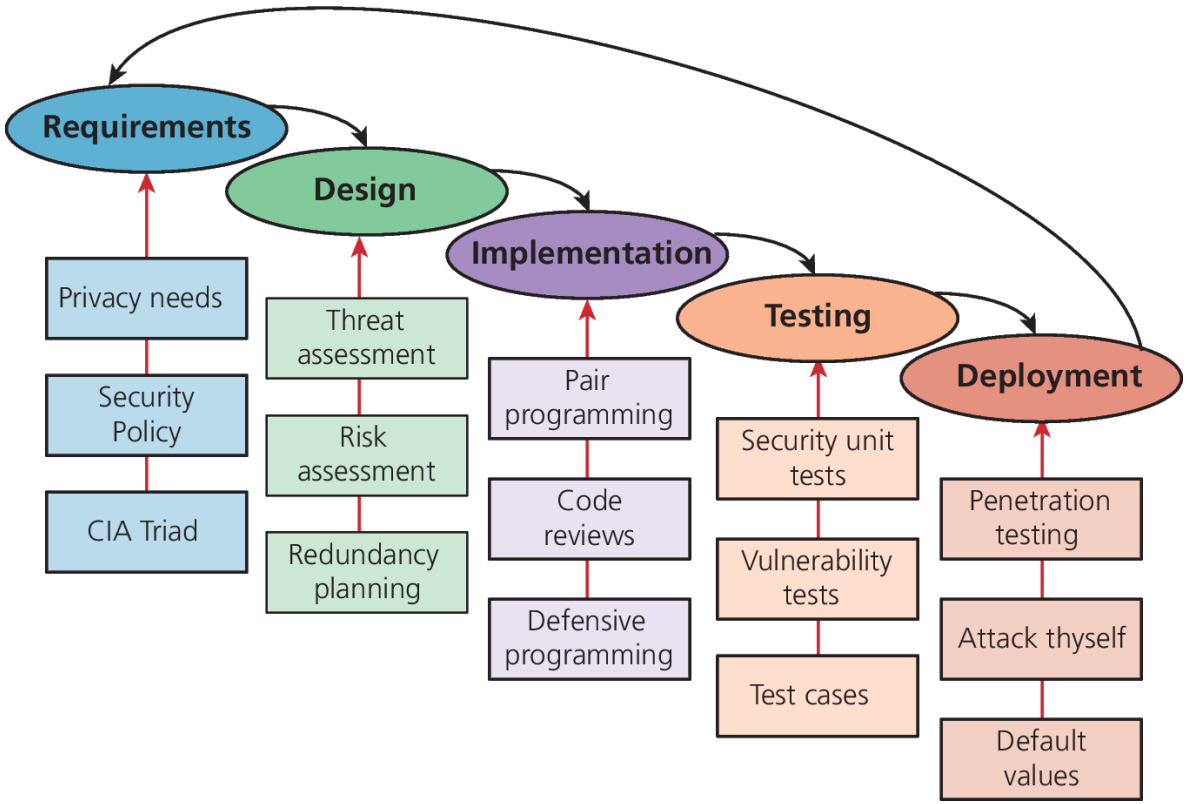
The best way to be prepared for the unexpected is to plan while times are good and thinking is clear. Some considerations include:

- **Admin Password Management**
- **Backups and Redundancy**
- **Geographic Redundancy**
- **Stage Mock Events**
- **Auditing**

Secure by Design

Secure by design is a software engineering principle that tries to make software better by acknowledging that there are malicious users out there and addressing it.

By continually distrusting user input (and even internal values) throughout the design and implementation phases, you will produce more secure software than if you didn't consider security at every stage.



Secure by Design (ii)

In a **code review** system, programmers must have their code peer-reviewed before committing it to the repository.

Unit testing is the practice of writing small programs to test your software as you develop it.

Pair programming is the technique where two programmers work together at the same time on one computer.

Security testing is the process of testing the system against scenarios that attempt to break the final system.

Secure by default aims to make the default settings of a software system secure

Social Engineering

Social engineering is the broad term given to describe the manipulation of attitudes and behaviors of a populace, often through government or industrial propaganda and/or coercion.

No one would click a link in an email that said *click here to get a virus*, but they might click a link to *get your free vacation.*

Authentication Factors

To achieve both *confidentiality* and *integrity*, the user accessing the system must be who they purport to be.

Authentication is the process by which you decide that someone is who they say they are and therefore permitted to access the requested resources.

Authentication factors are the things you can ask someone for in an effort to validate that they are who they claim to be.

- Knowledge factors
- Ownership factors
- Inherance Factors

Single versus Multifactor Authentication

Single-factor authentication is the weakest and most common category of authentication system where you ask for only one of the three factors.

Multifactor authentication is where two distinct factors of authentication must pass before you are granted access.

To enhance authentication, one should use multiple factors rather than multiple instances of the same factor.

Approaches to Web Authentication

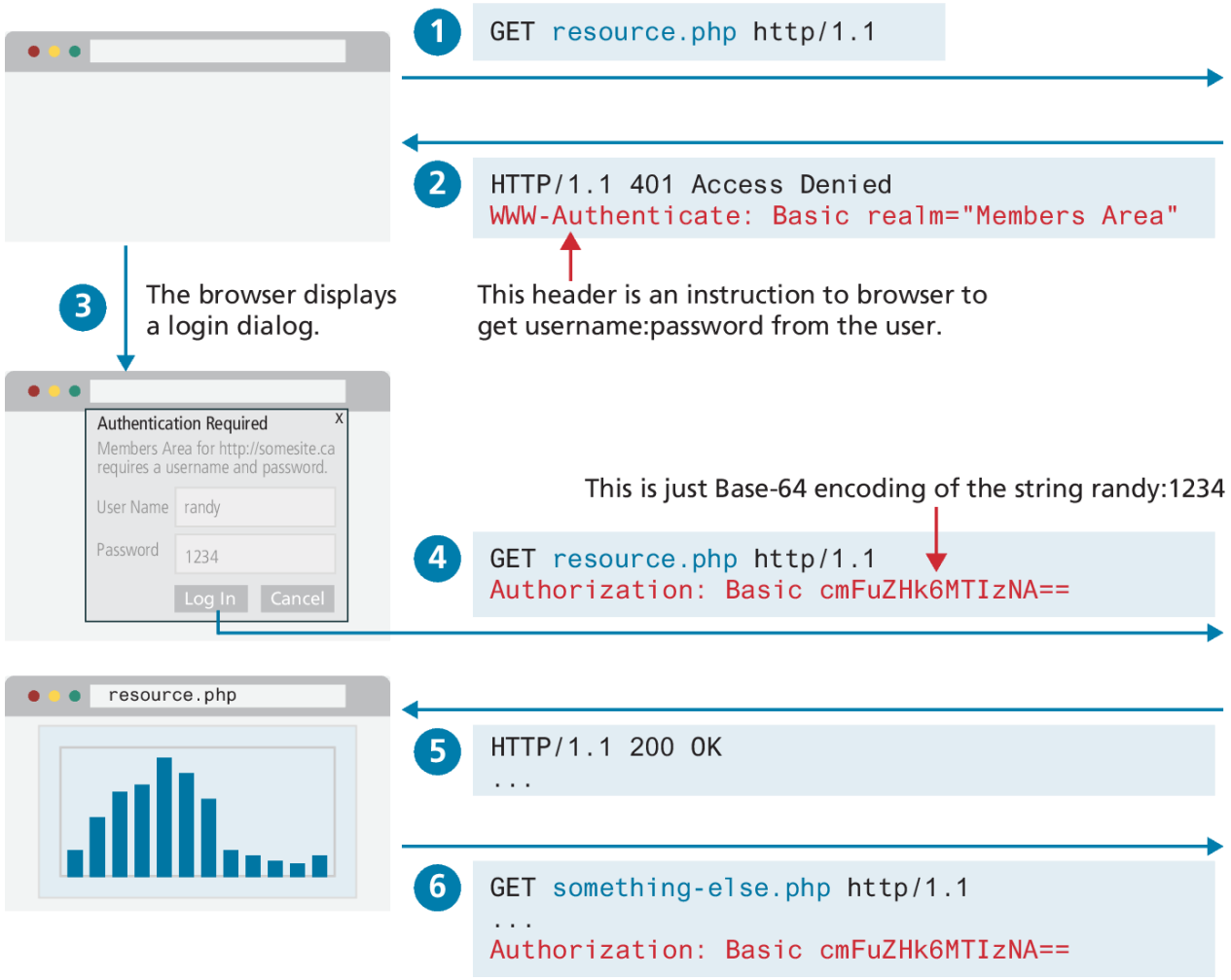
In web applications, there are four principle strategies used for authentication:

- Basic HTTP Authentication
- Form-Based Authentication
- Token HTTP Authentication
- Third-Party Authentication

Basic HTTP Authentication

HTTP Basic Authentication is a way for the server to indicate that a username and password is required to access a resource.

This approach is sometimes referred to as an example of challenge-response authentication, in that the server provides a "challenge" and the client has to *immediately* provide a response.

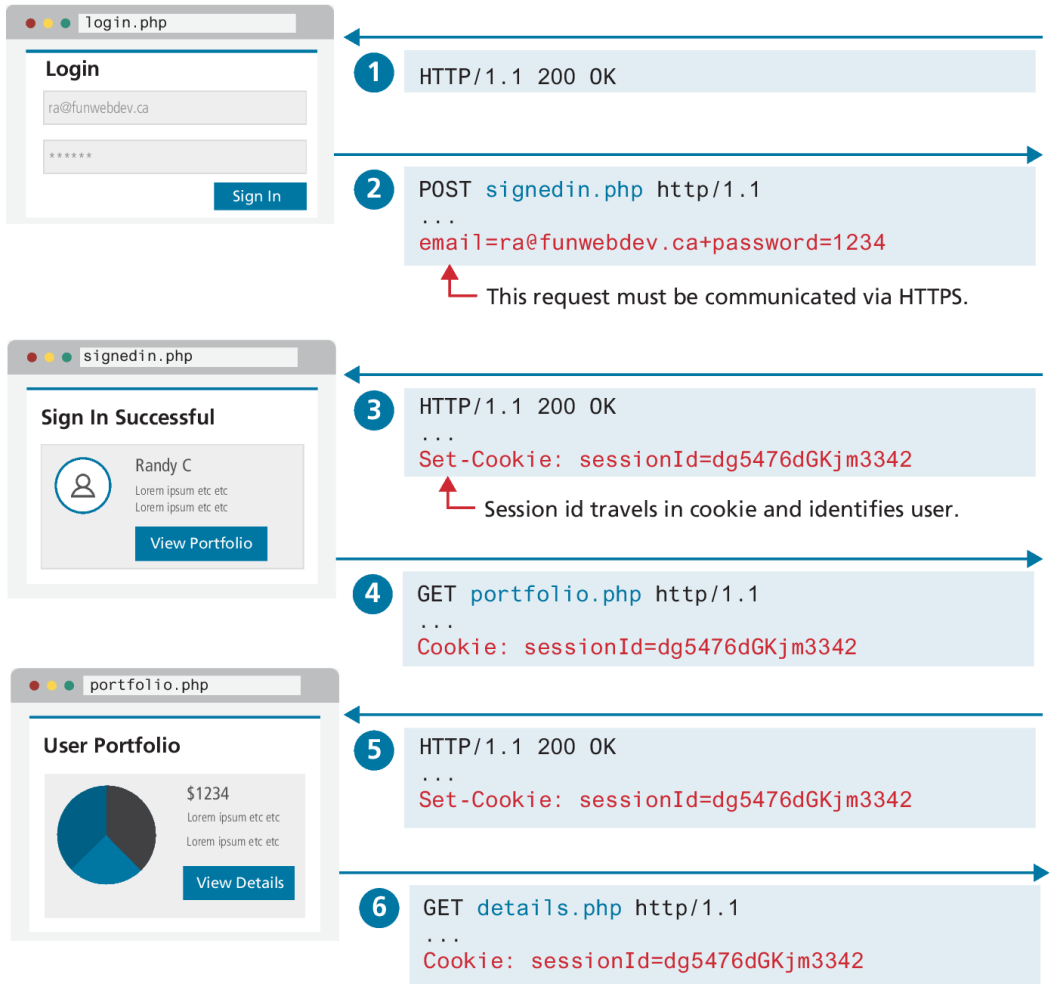


Form-Based Authentication

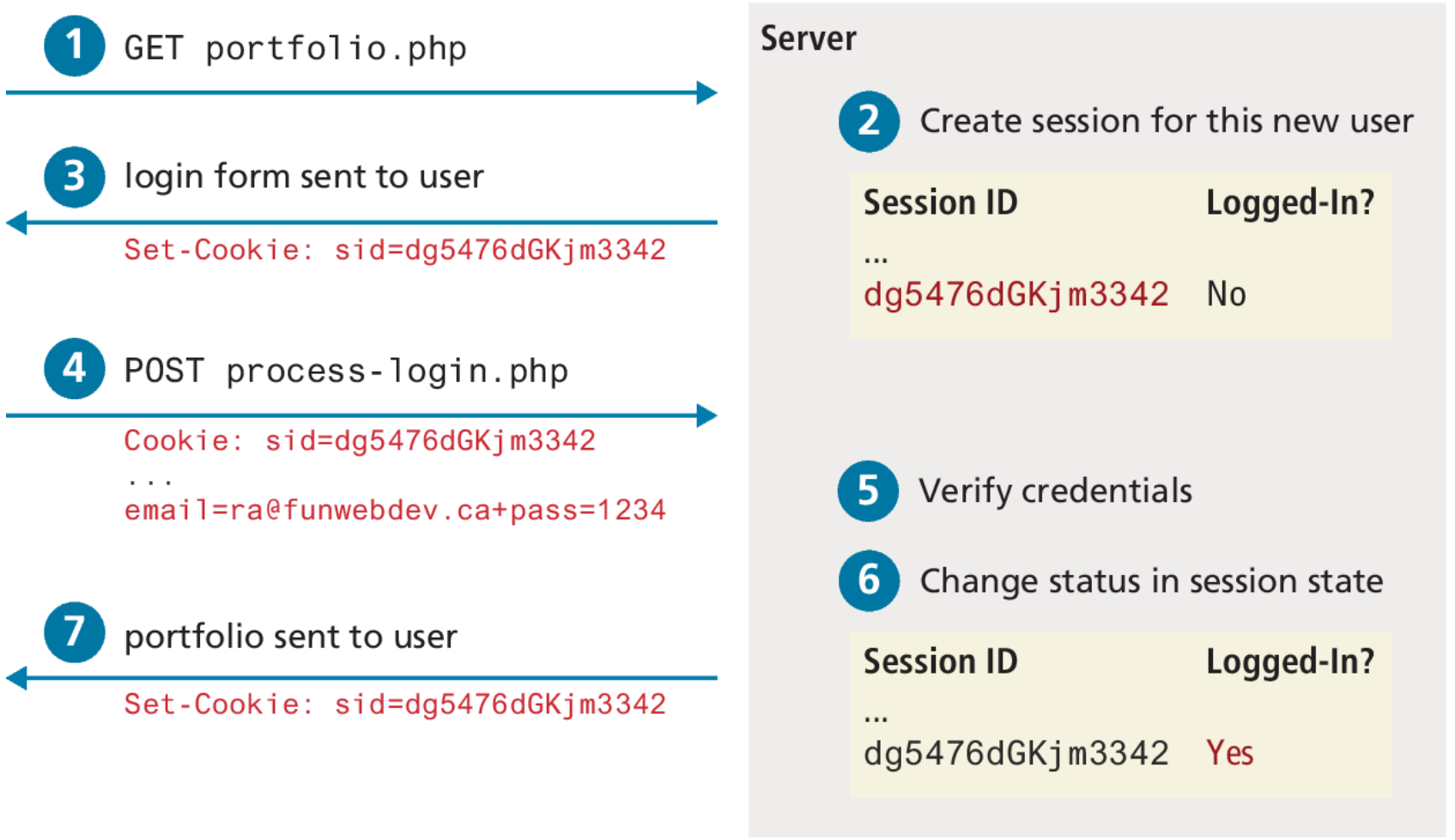
When secure communication is needed, websites generally use some form of **form-based authentication**

An HTML form is presented to the user, and the login credential information is sent via regular HTTP POST.

Form authentication keeps track of the user's login status. The example in the diagram is using a session cookie and must check credentials.



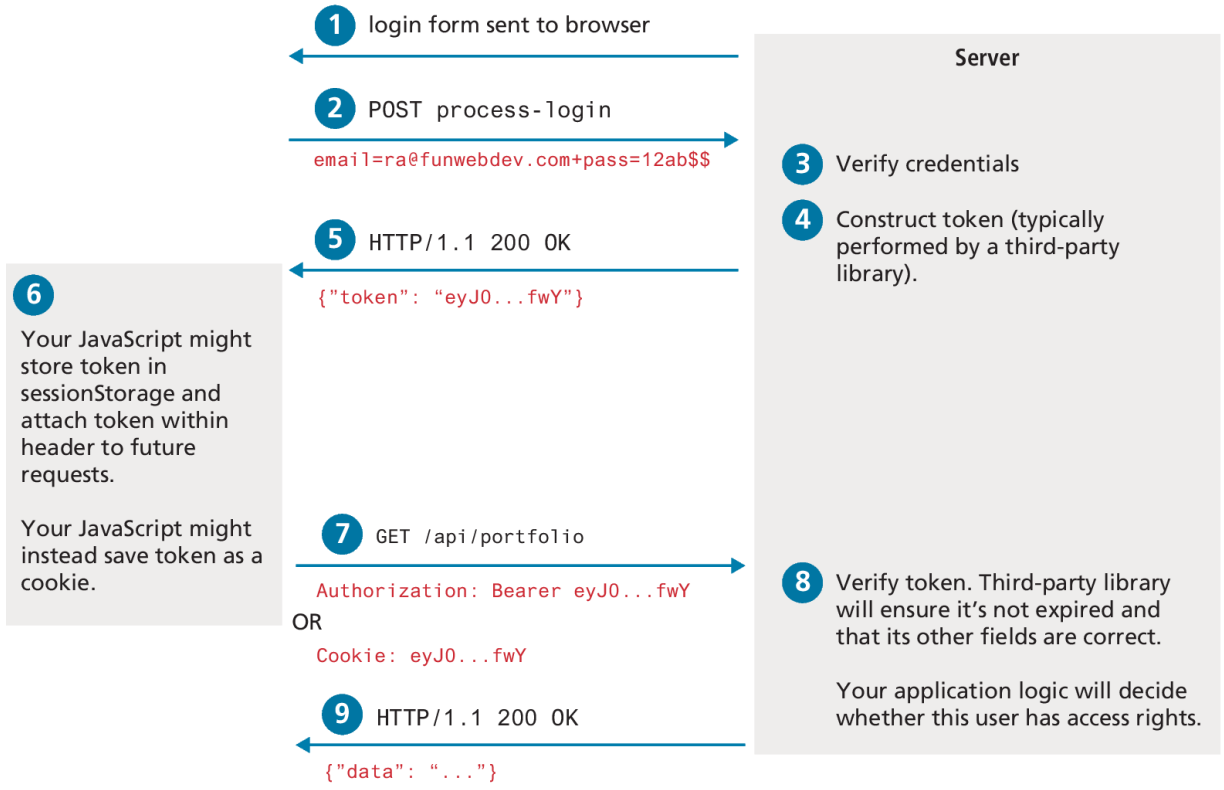
Managing HTTP Form login status



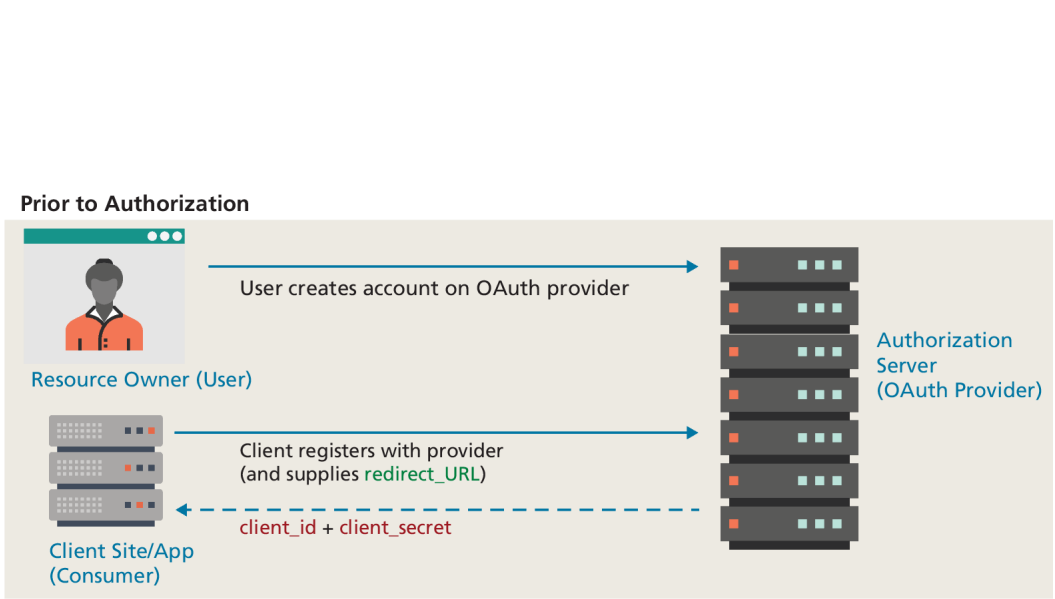
HTTP Token Authentication

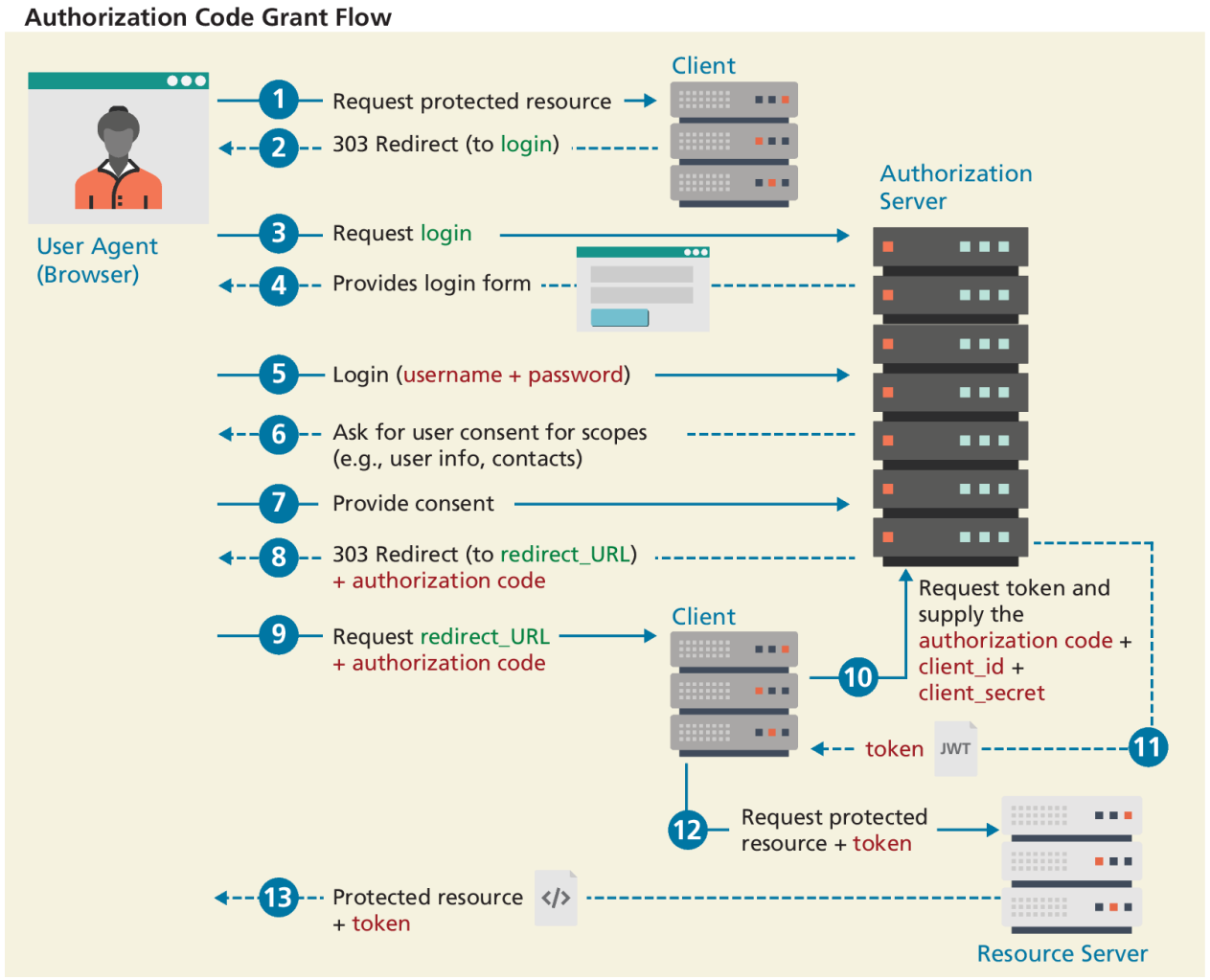
HTTP Token Authentication (also known more formally as **Bearer Authentication**) is a form of HTTP authentication that is commonly used in conjunction with form authentication, as well as with APIs and other services without a user interface

Token authentication provides a way to implement **stateless authentication**.



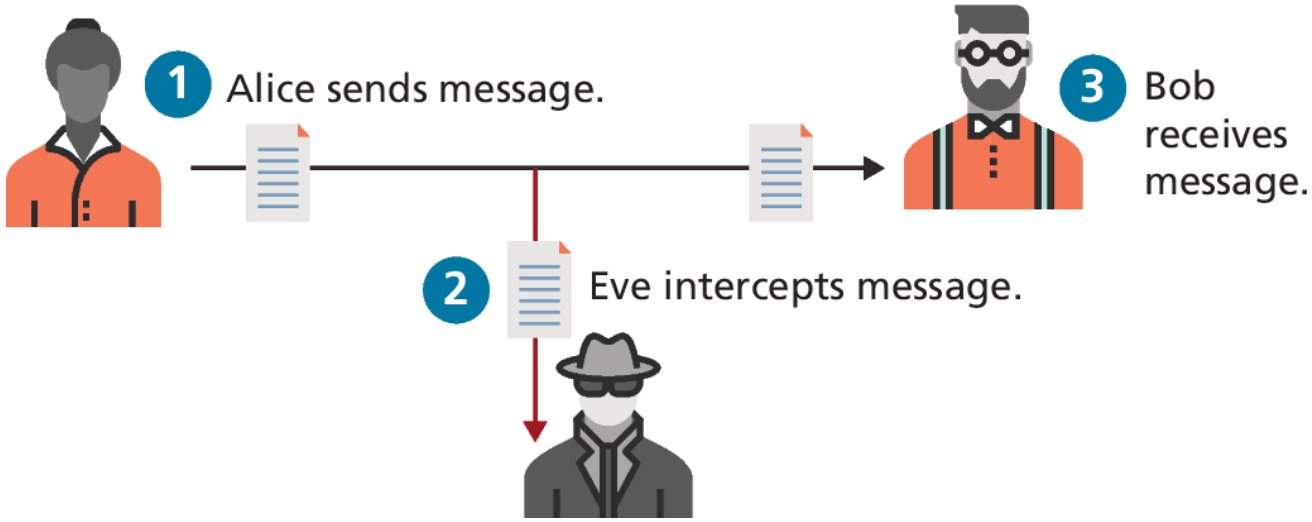
Third-Party Authentication





Cryptography

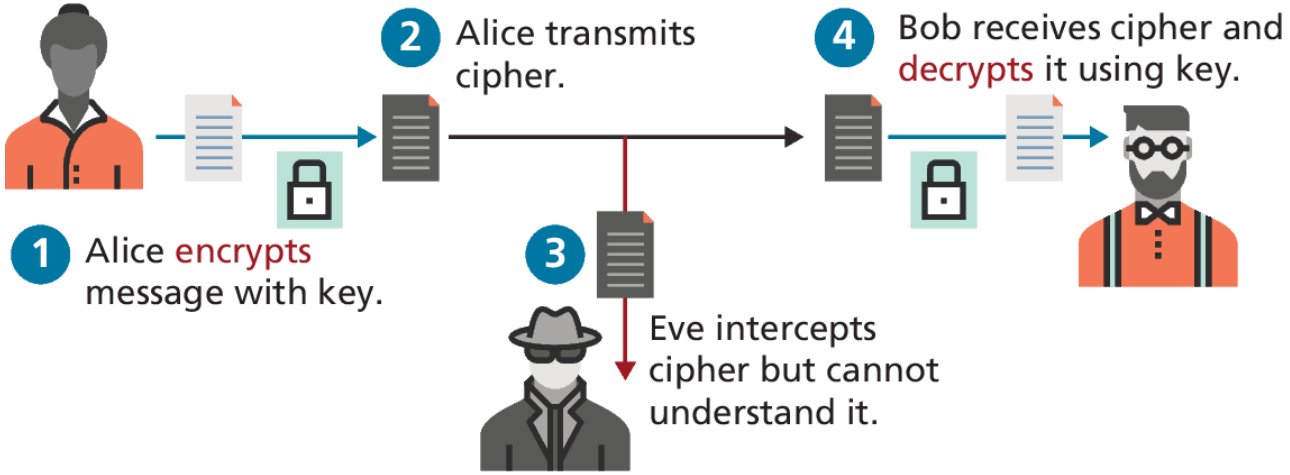
At a basic level we are trying to get a message from one actor (we will call her **Alice**), to another (**Bob**), without an eavesdropper (**Eve**) intercepting the message (as shown in Figure 16.9)



Cryptography (ii)

A **cipher** is a message that is scrambled so that it cannot easily be read, unless one has some secret knowledge. This secret is usually referred to as a **key**.

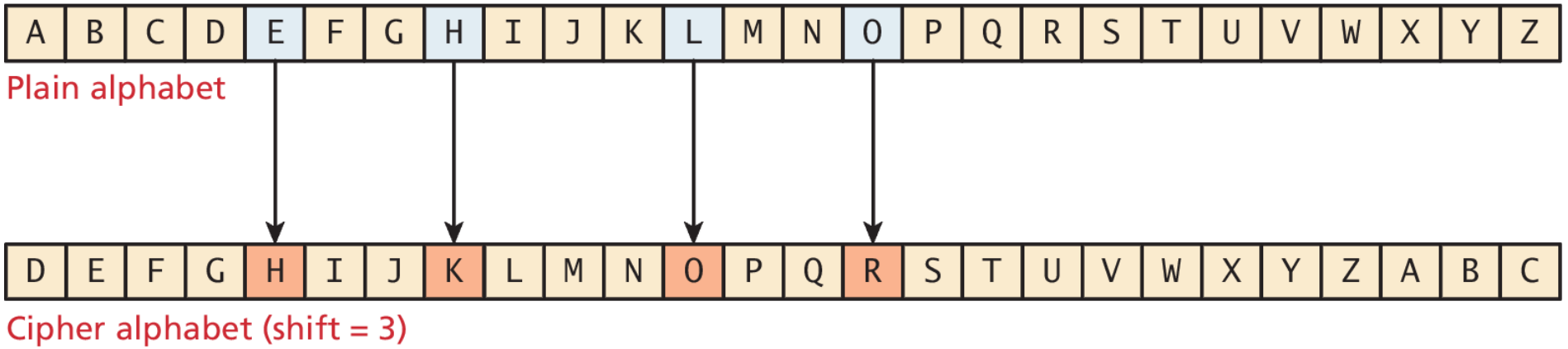
Alice encrypts the message (**encryption**) and Bob, the receiver, decrypts the message (**decryption**), both using their keys.



Substitution Ciphers

A **substitution cipher** is one where each character of the original message is replaced with another character according to the encryption algorithm and key.

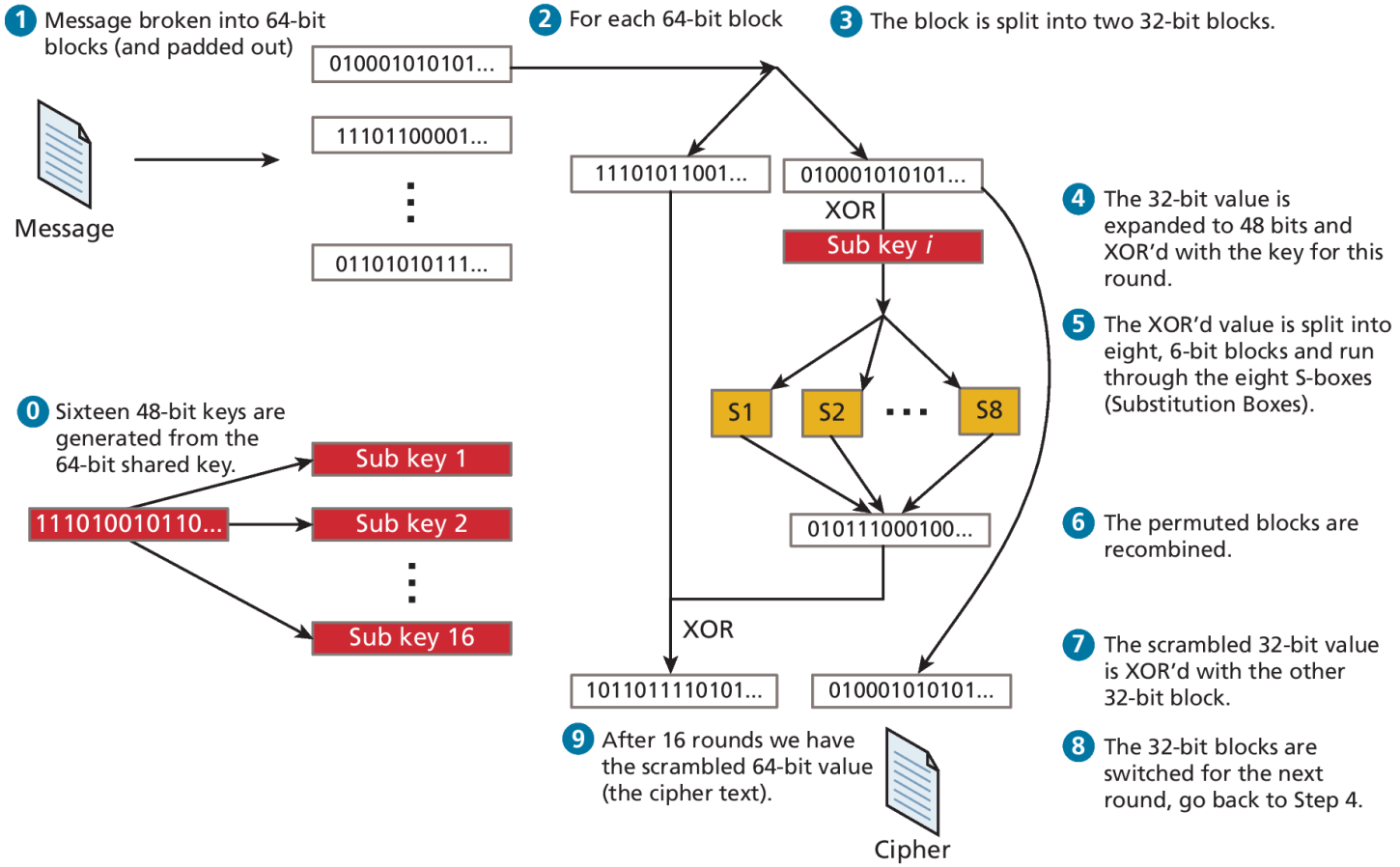
The Caesar cipher is a substitution cipher where every letter of a message is replaced with another letter, by shifting the alphabet over an agreed number (from 1 to 25). The message HELLO, for example, becomes KHOOR when a shift value of 3 is used



Modern Block Ciphers

Block ciphers encrypt and decrypt messages using an iterative replacing of a message with another scrambled message using 64 or 128 bits at a time (the block)

The Data Encryption Standard (DES) and its replacement, the Advanced Encryption Standard (AES) are two-block ciphers still used in web encryption today.



Public Key Cryptography

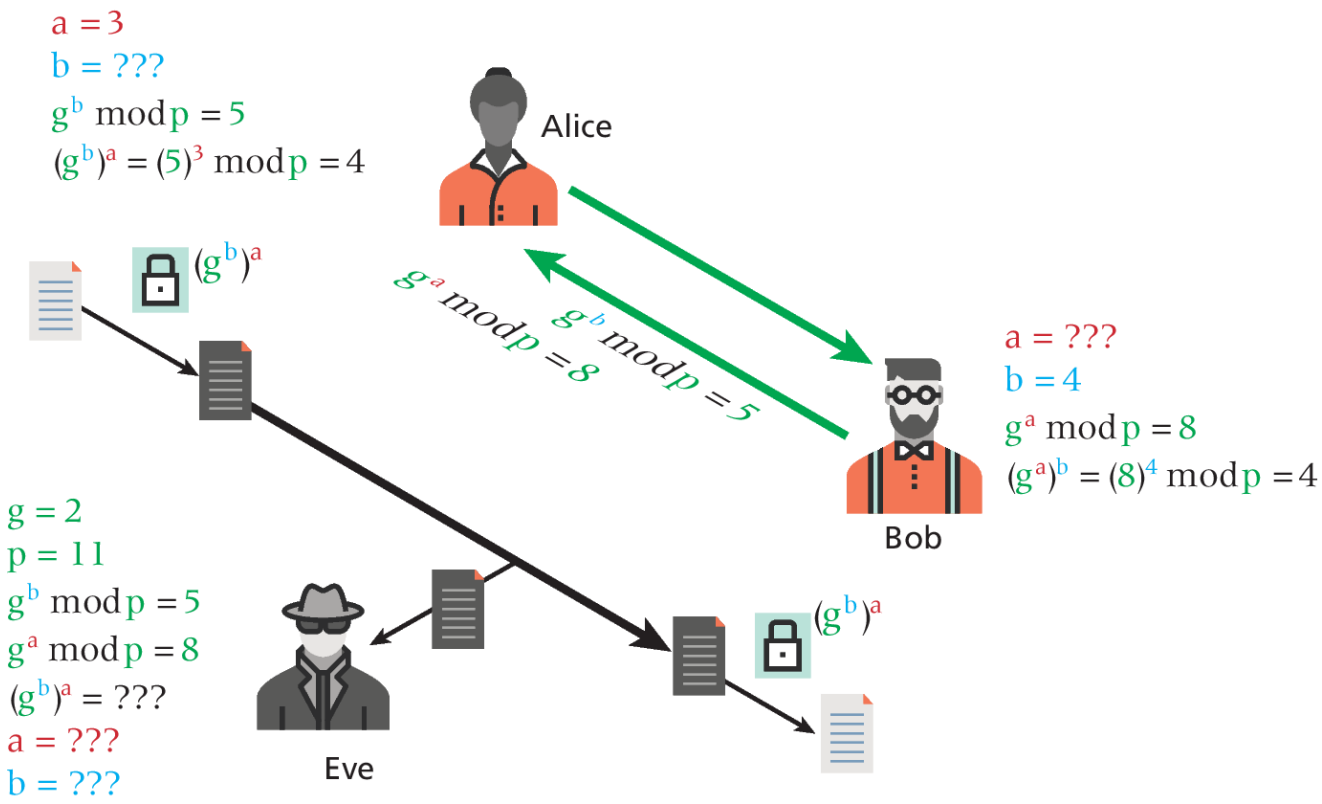
All of the ciphers we have covered thus far use the same key to encode and decode, so we call them **symmetric ciphers**. The problem is that we have to have a shared private key!

Public key cryptography (or **asymmetric cryptography**) solves the problem of the secret key by using two distinct keys: a public one, widely distributed and another one, kept private.

Diffie-Hellman key exchange are accessible to a wide swath of readers, and subsequent algorithms (like RSA) apply similar thinking but with more complicated mathematics.

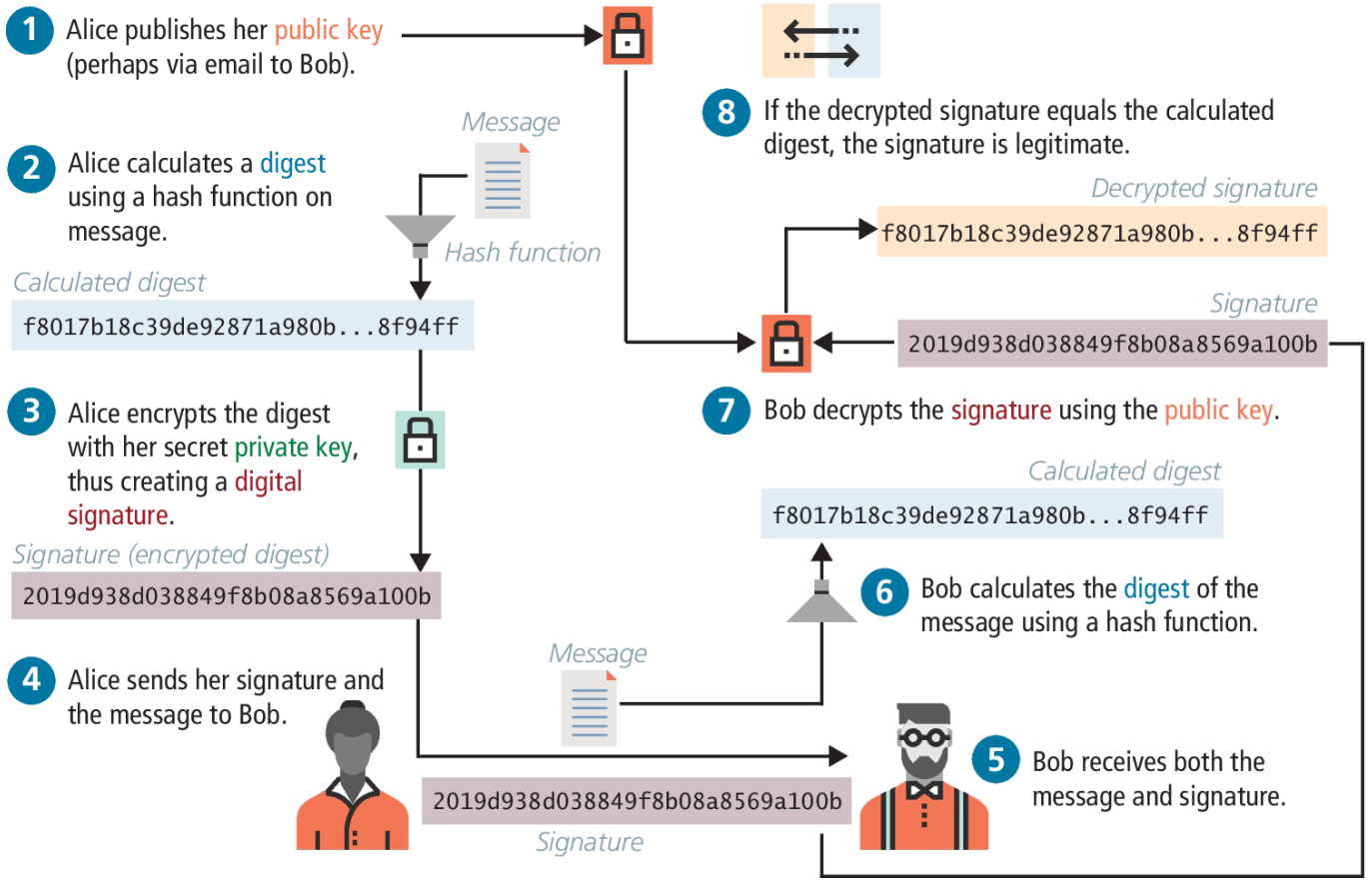
Diffie-Hellman Key Exchange

The essence of the key exchange is that this g^{ab} can be used as a *symmetric* key for encryption, but since only g^a and g^b are transmitted the symmetric key isn't intercepted.



Digital Signatures

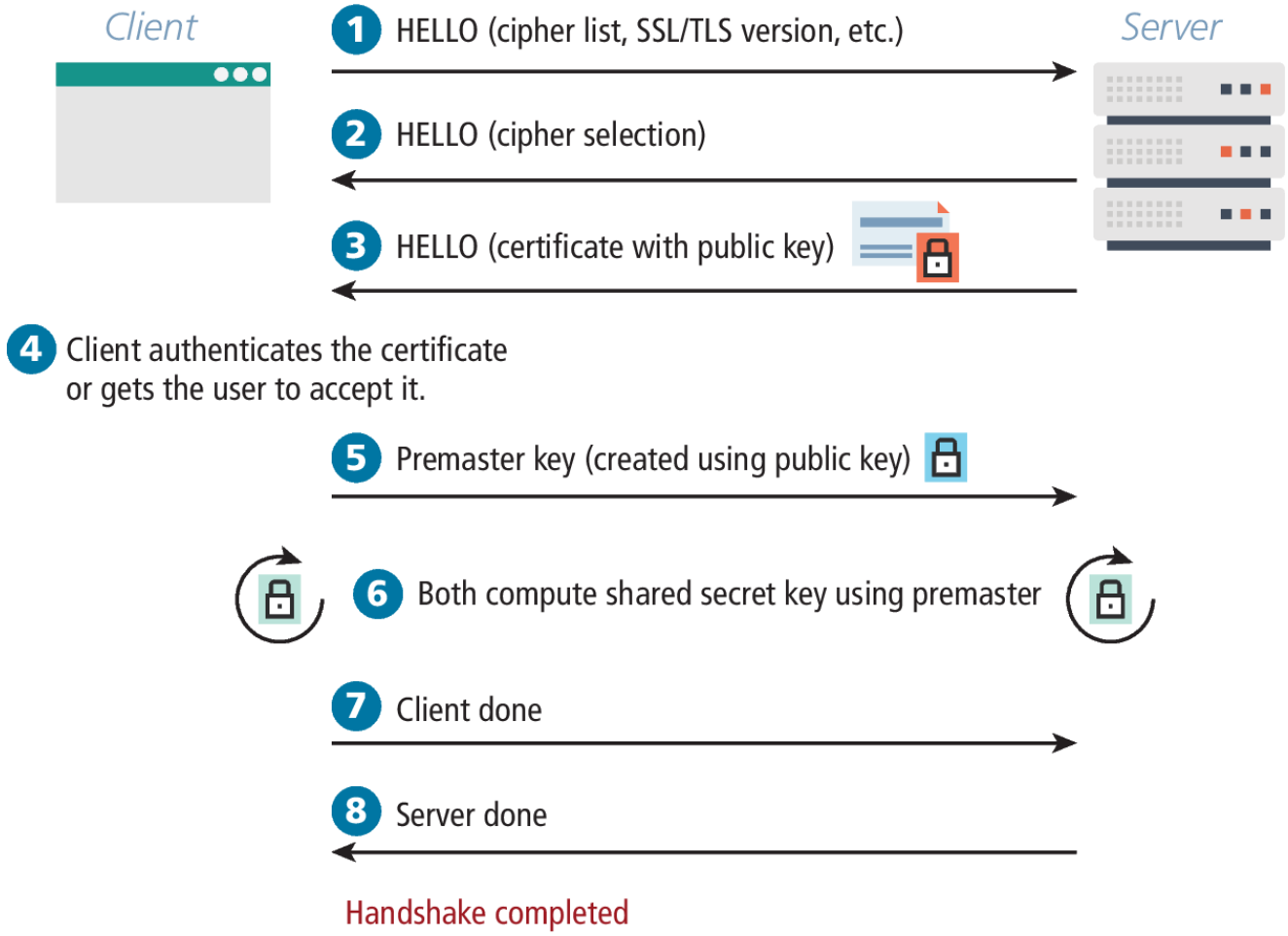
A **digital signature** is a mathematically secure way of validating that a particular digital document was created by the person claiming to create it (authenticity), was not modified in transit (integrity), and to prevent sender from denying that she or he had sent it (nonrepudiation).



SSL/TLS Handshake

The client initiates the handshake by sending the time, the version number, and a list of cipher suites its browser supports to the server.

The server, in response, sends back which of the client's ciphers it wants to use as well as a **certificate**, which includes a public key.

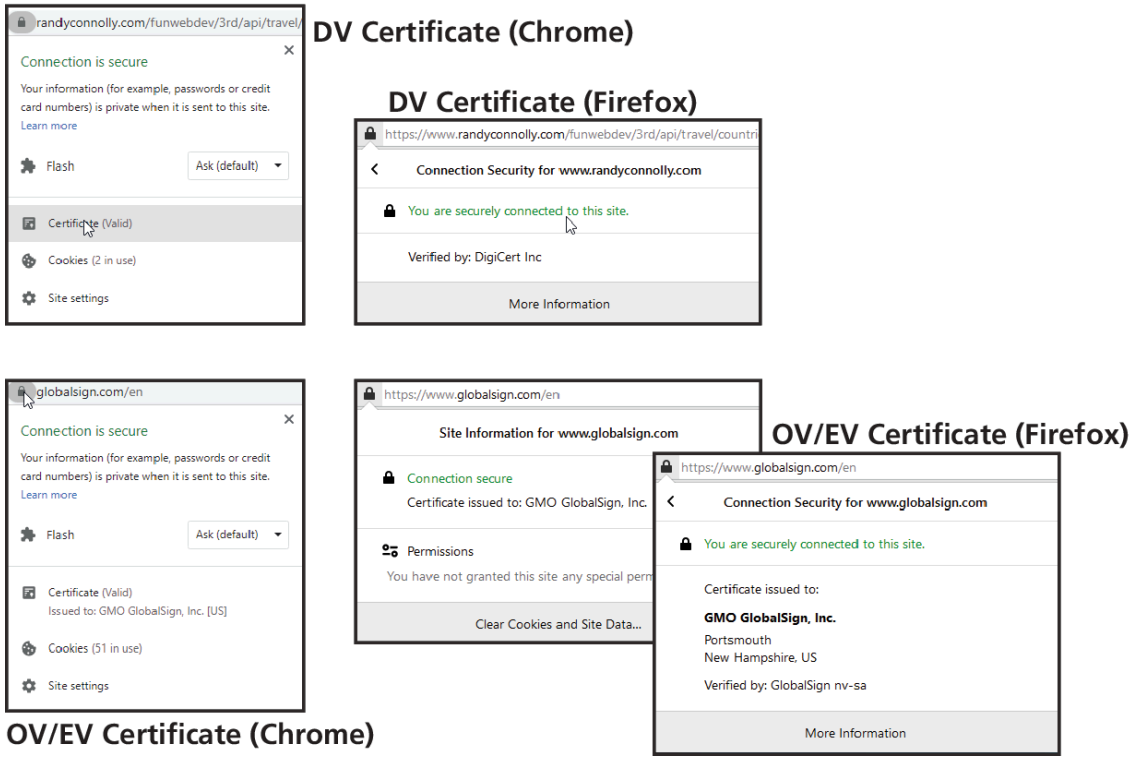


Certificates and Authorities

A **Certificate Authority** (CA) allows users to place their trust in the certificate since a trusted, independent third party signs it.

There are three types of SSL certificates that can be purchased:

- **Domain-validated certificates**
- **Organization-validated certificates**
- **Extended-validation certificates**



Domain-Validated (DV) Certificates

This is the most affordable option. Most Cas will only verify the email listed in the whois registration database (see Chapter 2) via a confirmation link.

As a consequence, the process of obtaining the certificate is very fast.

Many CAs also offer wildcard certificates or even multi-domain certificates that allow an organization to secure a wider range of domains they own.

Organization-Validated (OV) Certificates

With these certificates, the CA takes additional steps to verify the identity of the organization seeking the certificate.

While it will perform the same domain verification as with domain-validated certificates, it also typically requests a variety of business documents, such as a government license, bank statement, or legal incorporation records. As a consequence, this type of certificate typically takes several days and is more expensive.

Extended-Validation (EV) Certificates

These are similar to the organization-validated certificatess, but have even stricter requirements around the documentation that needs to be provided by the purchaser.

The rationale for choosing this option is similar to that of the OV: it’s to improve the trust of the end user.

Migrating to HTTPS

Coordinating the migration of a website can be a complex endeavor involving multiple divisions of a company. In addition to business considerations, there are also some technical considerations in migrating to HTTPS.

- **Mixed Content**
- **Redirects from Old Site**
- **Preventing HTTP Access**

Mixed Content

In order to fully address a transition from HTTP to HTTPS, developers have to consider every place a HTTP reference exists in their code

Hardcoded links (which are bad style—and now we see why) should be replaced with relative links that easily transform according to the protocol being used. These links might include the following:

- Internal links within the site.
- External links to frameworks delivered through a CDN.
- Any links or references generated by server code that might include a hardcoded http.

Redirects from Old Site

Once you move your site over to HTTPS, there likely be links remaining from third-party sites to your former HTTP URLs and it's important that that such links still work.

To enable such behavior for every possible resource, both Apache and Nginx server provide mechanisms for redirecting HTTP requests for a resource to HTTPS requests.

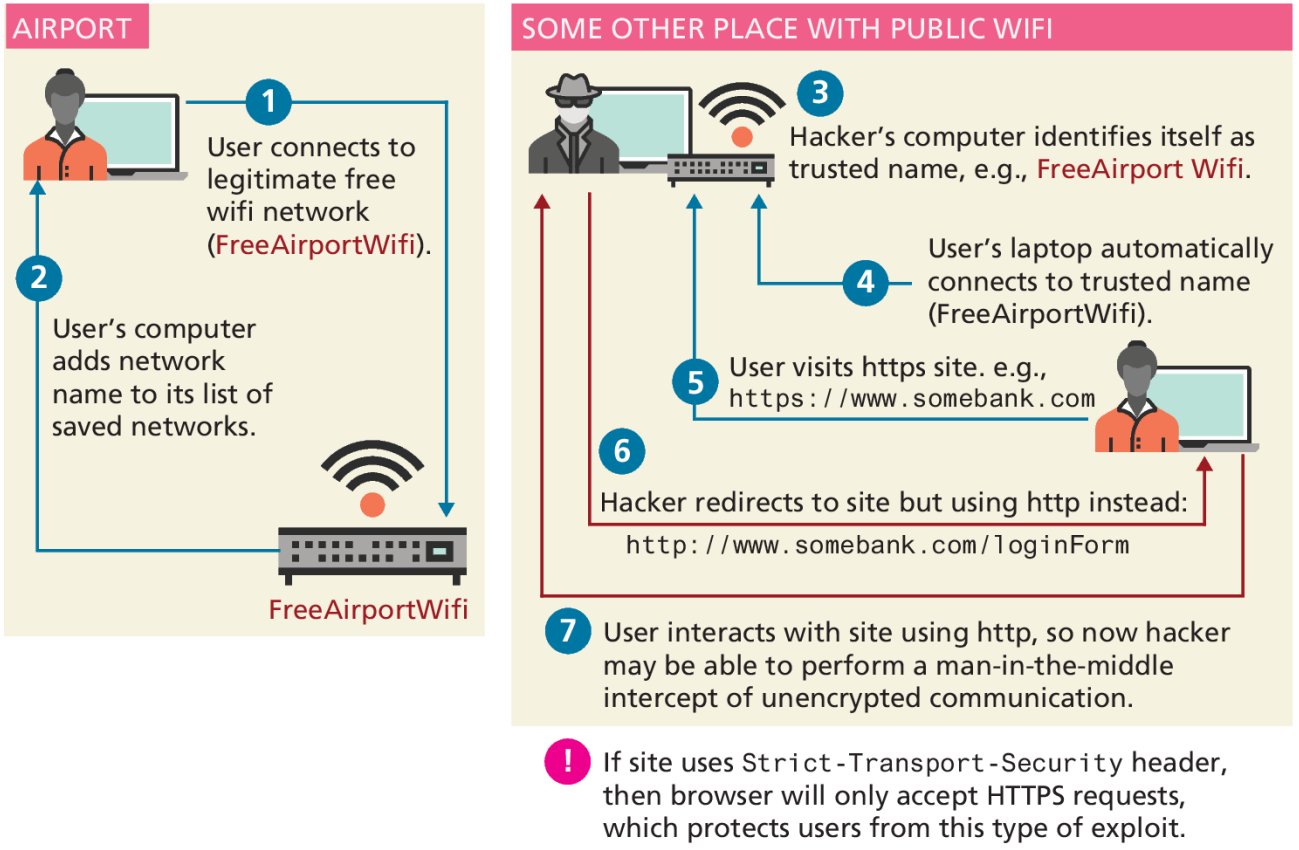
For instance, in Apache, the following two lines will send a 301 code and the new link location on https.

```
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

Preventing HTTP Access

Once your site has added HTTPS capabilities, it often makes sense to prevent users from accessing your site resources using HTTP.

In this example, the hacker will be able to redirect the user from HTTPS to HTTP, thereby having unencrypted access to the user's experience.



Credential Storage

When (not if) you are breached, what data will the attacker have access to?

SQL table structures that store the username and password in the table are bad form

User ID	Username	Password
1	ricardo	password
2	randy	password

```
function insertUser($username,$password){
    $pdo = new PDO(DBCONN_STRING,DBUSERNAME,DBPASS);
    $sql = "INSERT INTO Users (Username,Password)
        VALUES('?,?')";
    $smt = $pdo->prepare($sql);
    $smt->execute(array($username,$password));

}
function validateUser($username,$password){
    $pdo = new PDO(DBCONN_STRING,DBUSERNAME,DBPASS);
    $sql = "SELECT UserID FROM Users WHERE Username=? AND
            Password=?";
    $smt = $pdo->prepare($sql);
    $smt->execute(array(($username,$password));
    if($smt->rowCount())){
        return true;
    }
        return false;
}
```

LISTING 16.1 First approach to storing passwords (very insecure)

Credential Storage – Hash Function

Instead of storing the password in plain text, a better approach is to store a **hash** of the data, so that the password is not discernable.

UserID	Username	Password
1	ricardo	5f4dcc3b5aa765d61 d8327deb882cf99
2	randy	5f4dcc3b5aa765d61 d8327deb882cf99

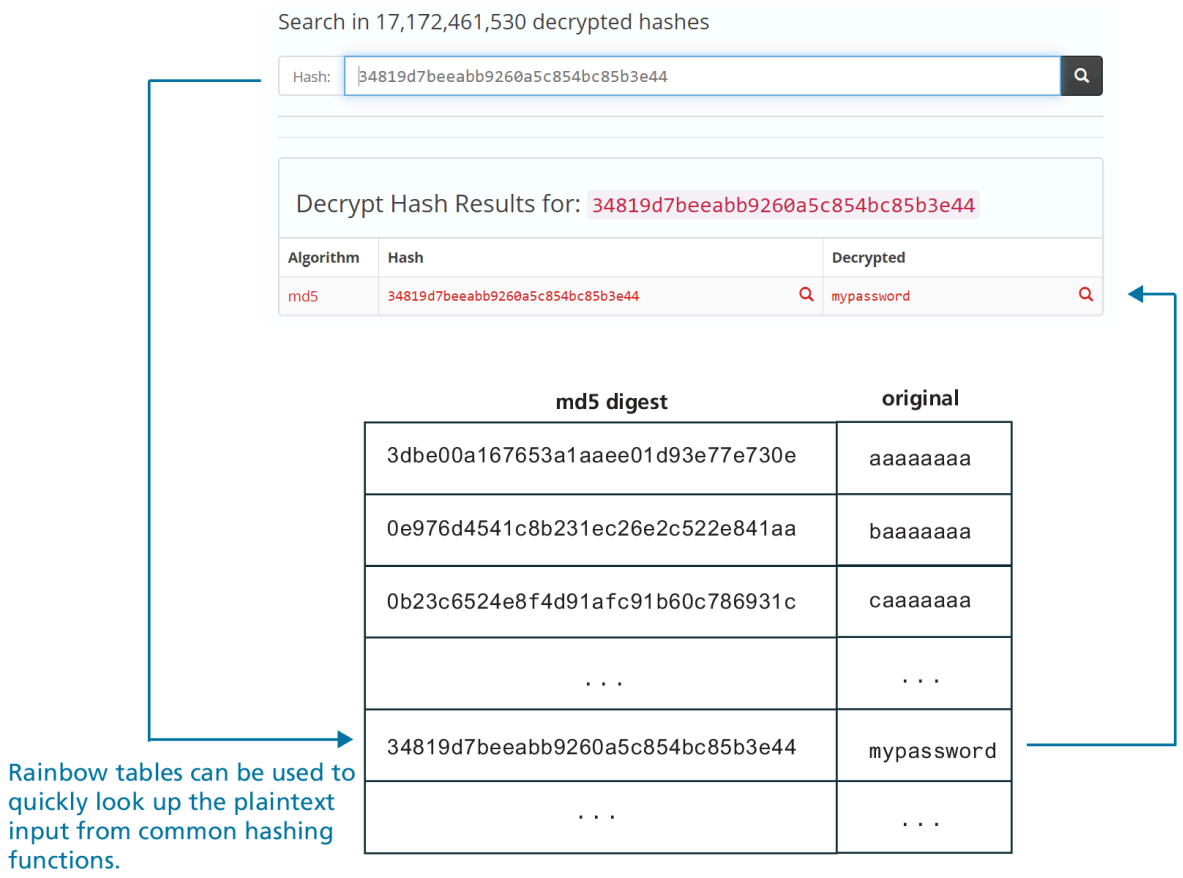
```
function insertUser($username,$password){
    $pdo = new PDO(DBCONN_STRING,DBUSERNAME,DBPASS);
    $sql = "INSERT INTO Users (Username,Password)
        VALUES('?,?')";
    $smt = $pdo->prepare($sql);
    $smt->execute(array($username,md5($password)));
}
function validateUser($username,$password){
    $pdo = new PDO(DBCONN_STRING,DBUSERNAME,DBPASS);
    $sql = "SELECT UserID FROM Users WHERE Username=? AND
            Password=?";
    $smt = $pdo->prepare($sql);
    $smt->execute(array($username,md5($password)));    if($smt-
>rowCount()){
        return true;
    }
        return false;
}
```

LISTING 16.2 Second approach to storing passwords (better but still insecure)

Rainbow Tables

Rainbow tables are massive generated tables for common words and phrases that allow anyone who has access to the digest to quickly look up the original password

As a consequence, storing the MD5 digest of just the password is *not* recommended.

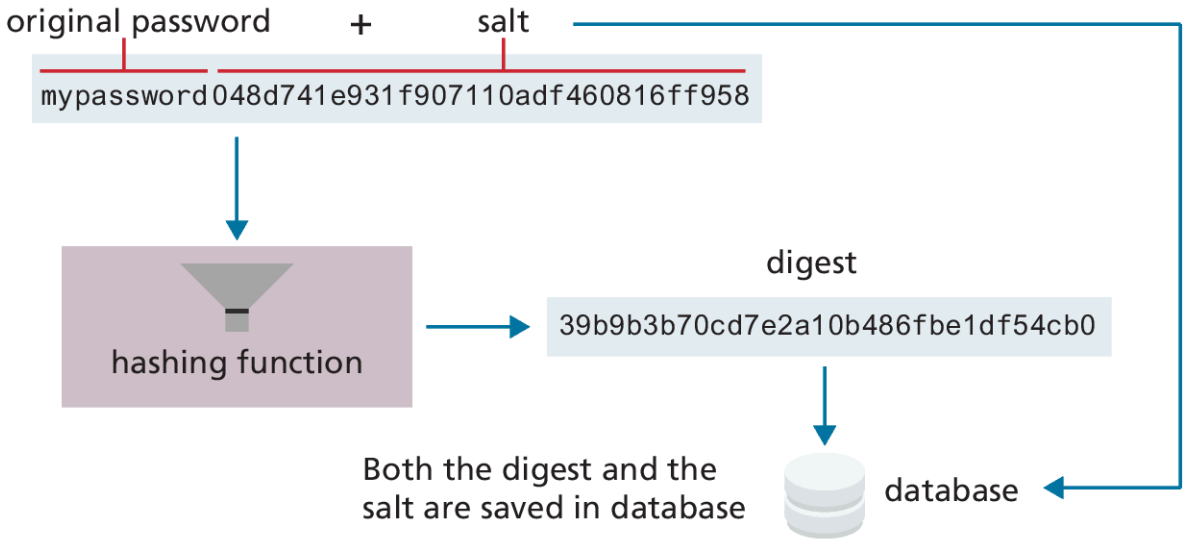


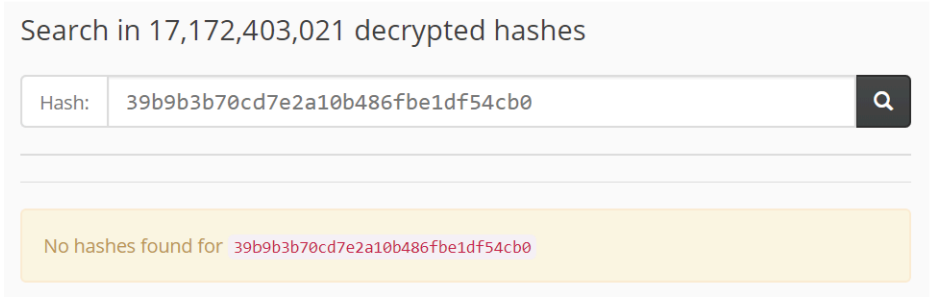
Salting the Hash

The solution to the rainbow table problem is to add some unique noise to each password, thereby lengthening the password before it is hashed.

The technique of adding some noise to each password is called **salting** the password.

Slow hash, using bcrypt even better





By making the password long and complex, salting generally protects leaked passwords against rainbow table decryption.

Monitor Your Systems

System Monitors allow you to preconfigure a system to check in on all your sites and servers periodically. Nagios, for example, comes with a web interface that allows you to see the status and history of your devices, and sends out notifications by email per your preferences.

Automating intrusion detection reads the log files and detects failed login attempts, then uses a history to determine the originating IP addresses to automatically block it in future

Common Threat Vectors

- **Brute-Force Attacks**
- **SQL Injection**
- **Cross-Site Scripting (XSS)**
- **Cross-Site Request Forgery (CSRF)**
- **Insecure Direct Object Reference**
- **Denial of Service**
- **Security Misconfiguration**

Brute-Force Attacks

In this attack, an intruder simply tries repeatedly guessing a password. For instance, an automated script might try looping through words in the dictionary or use combinations of words, numbers, and symbols.

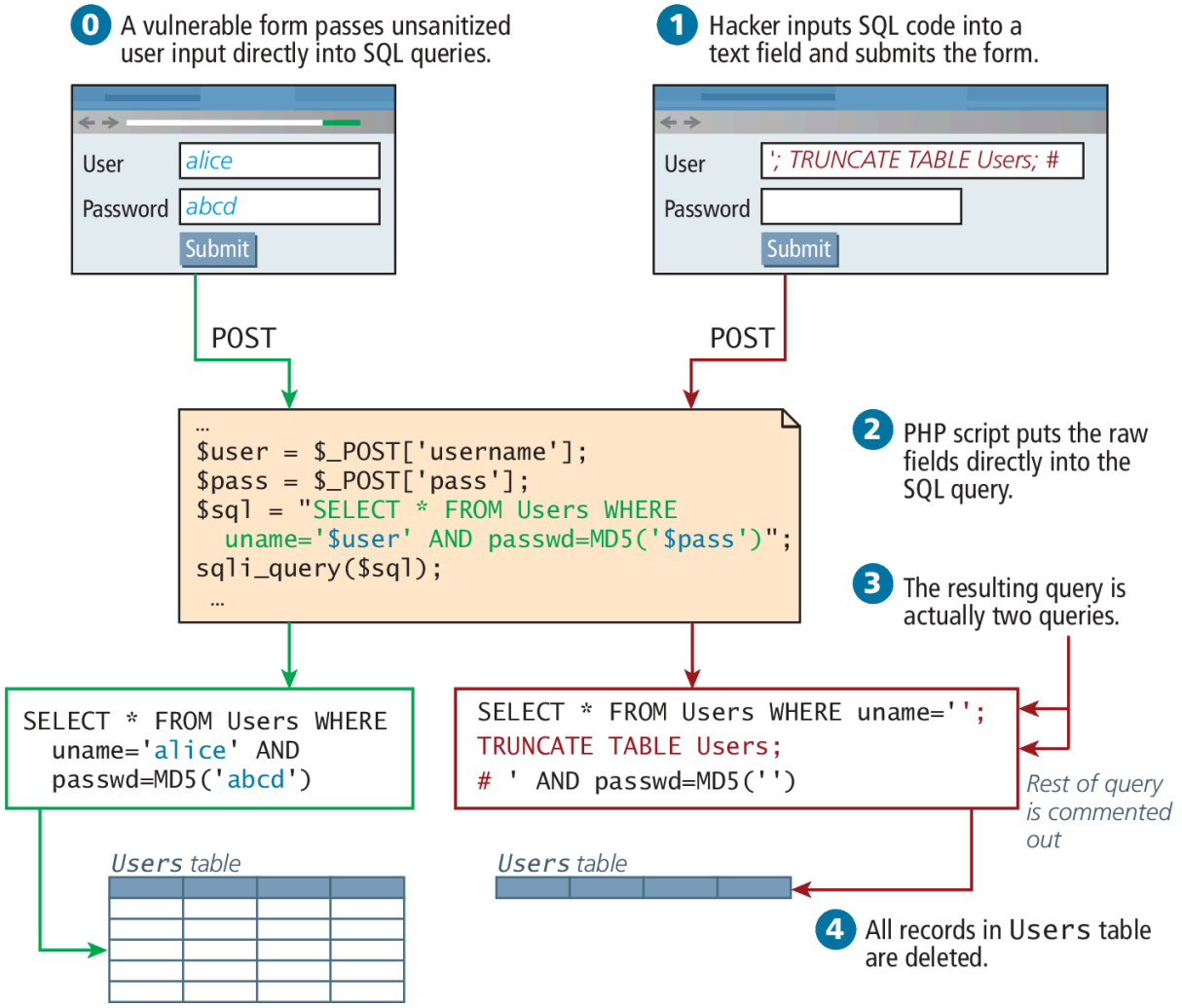
It it is important to throttle login attempts. One approach is to lock a user account after some set number of incorrect guesses. Another approach is to simply add a time delay between login attempts.

Automating intrusion detection can monitor logs to detect and block these attacks.

SQL Injection

SQL injection is the attack technique of entering SQL commands into user input fields in order to make the database execute a malicious query.

There are two ways to protect against such attacks: sanitize user input, and apply the least privileges possible for the application's database user.



Defending against SQL Injection

To **sanitize** user before using it in a SQL query, you can apply sanitization functions and bind the variables in the query using parameters or prepared statements.

Despite the sanitization of user input, there is always a risk that users could somehow execute a SQL query they are not entitled to. A properly secured system only assigns users and applications the privileges they need to complete their work, but no more.

Cross-Site Scripting (XSS)

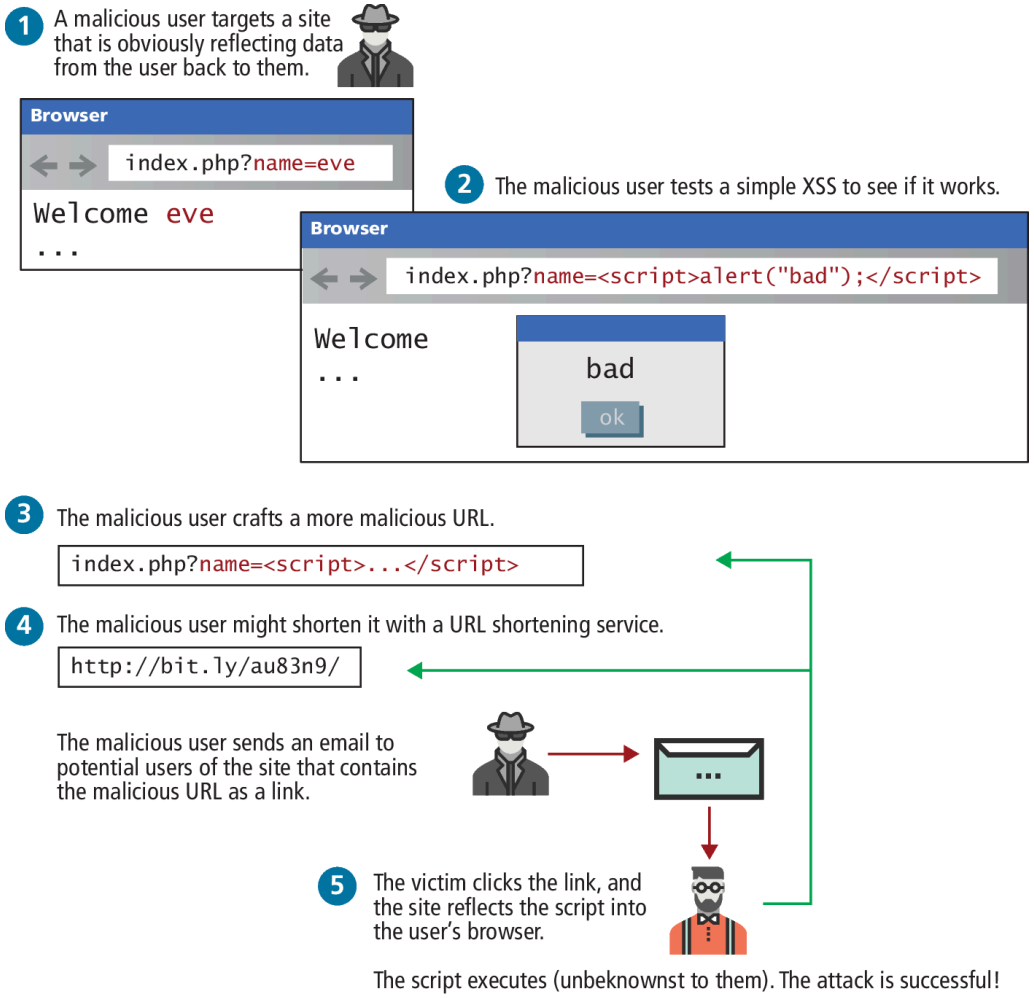
Cross-site scripting (XSS) refers to a type of attack in which a malicious script (JavaScript) is embedded into an otherwise trustworthy website.

In the original formulation for these type of attacks, a malicious user would get a script onto a page and that script would then send data to a malicious party, hosted at another domain (hence the **cross**, in XSS).

That problem has been partially addressed by modern browsers, which restricts script requests to the same domain. However, with at least 80 XSS attack vectors to get around those restrictions, it remains a serious problem.20 There are two main categories of XSS vulnerability: **Reflected XSS** and **Stored XSS**.

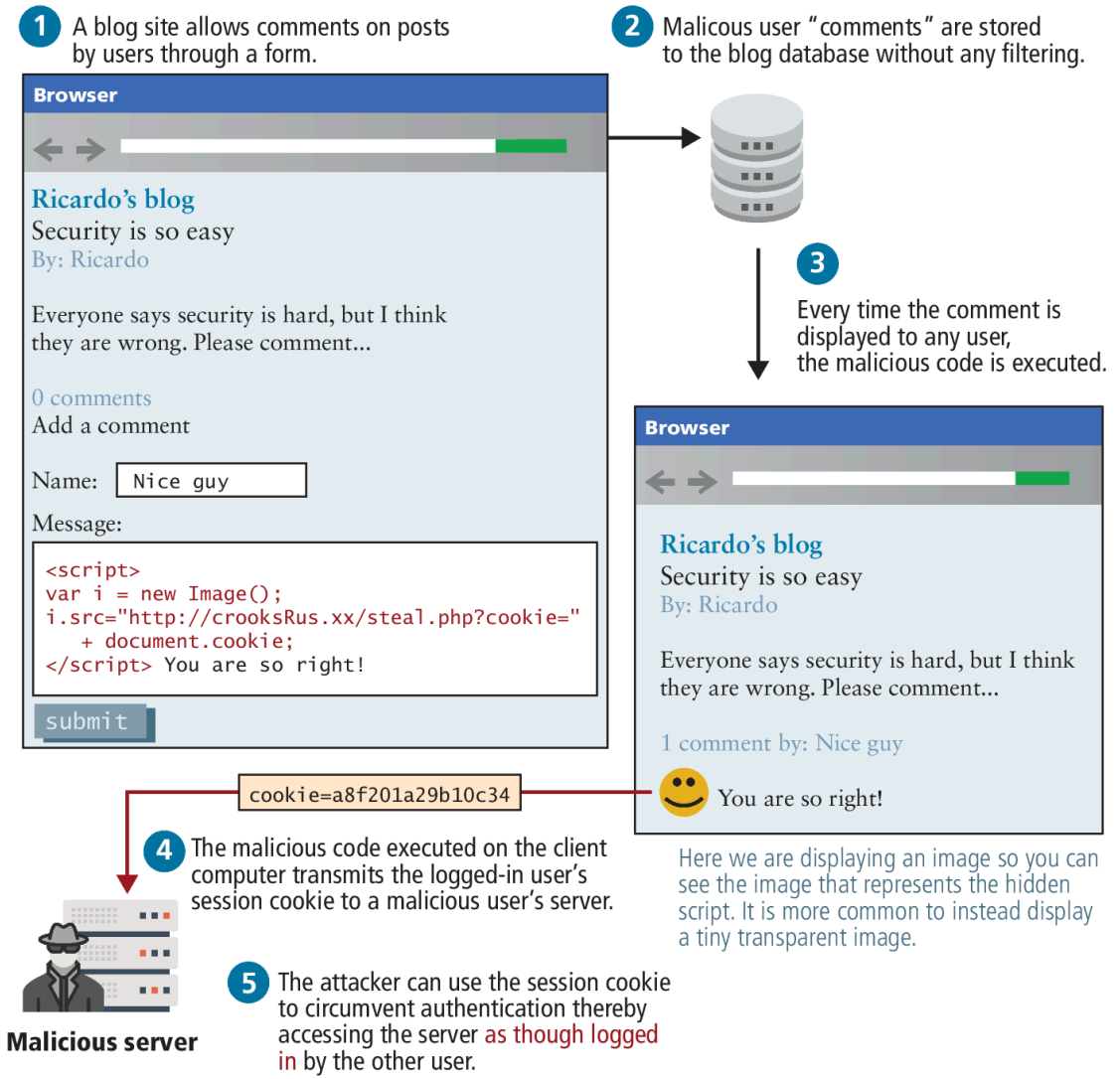
Reflected XSS

Reflected XSS (also known as nonpersistent XSS) are attacks that send malicious content to the server, so that in the server response, the malicious content is embedded.



Stored XSS

Stored XSS (also known as persistent XSS) is even more dangerous, because the attack can impact every user that visits the site.



Defending XSS Attack - Filtering

Sanitizing user input is crucial to preventing XSS attacks but attackers have gone far beyond using HTML script tags, and employ subtle tactics

- **Embedded attributes** use the attribute of a tag, rather than a <script> block
- **Hexadecimal/HTML encoding** embeds an escaped set of characters such as:

  %3C%73%63%72%69%70%74%3E%61%6C%65%72%74%28%22%68%65%6C%6C%6F%22%29%3B%3C%2F%73%63%72%69%70%74%3E

  instead of <script>alert("hello");</script>.

Most significant frameworks such as React or EJS provide built-in sanitization when outputting content. A library such as the opensource HTMLPurifier from http://htmlpurifier.org/ allows you to easily remove a wide range of dangerous characters

Escape Dangerous Content

Even if malicious content makes its way into your database, there are still techniques to prevent an attack from being successful.

Escaping content is a great way to make sure that user content is never executed, even if a malicious script was uploaded. This technique relies on the fact that browsers don't execute escaped content as JavaScript, but rather interpret it as text. Ironically, it uses one of the techniques the hackers employ to get past filters.

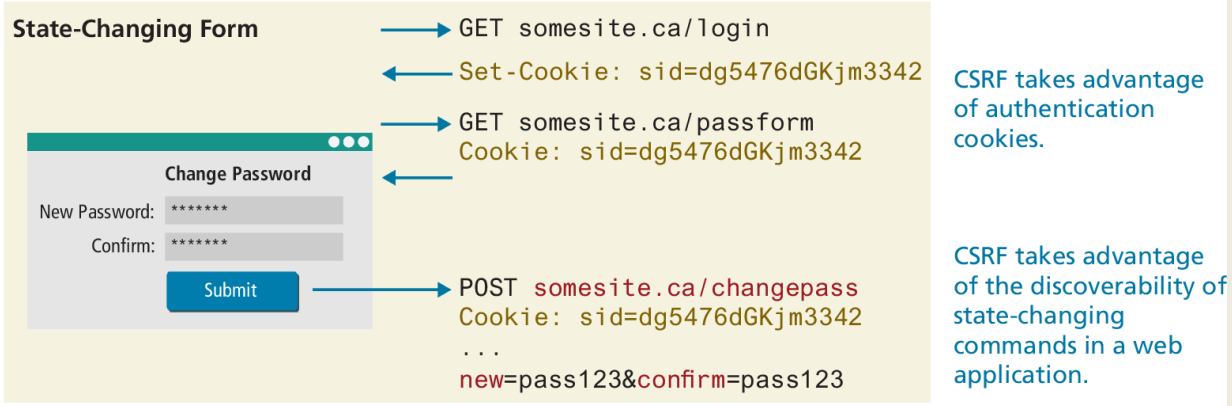
That means even if the malicious script did get stored, you would escape it before sending it out to users, so they would receive the following:

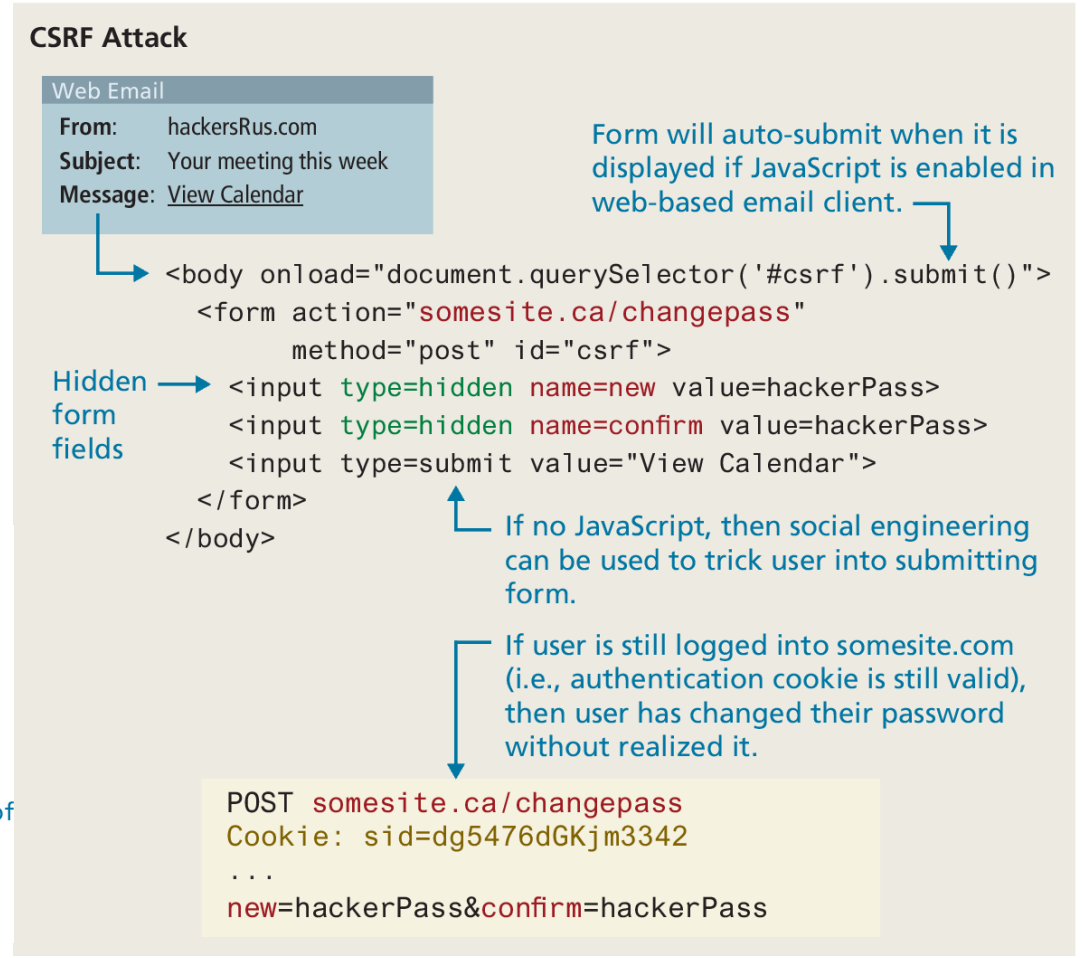
<script>alert("hello");</script>

The browsers seeing the encoded characters would translate them back for display, but will not execute the script! Instead your code would appear on the page as text.

Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a type of attack that forces users to execute actions on a website in which they are authenticated.





Defending against CSRF

Regardless of whether one uses tokens or cookies, the most common way to prevent CSRF attacks is to add a **one-time use CSRF token** to any state-changing form via a hidden field:

```
<input type="hidden" name="csrf-token" value="lR4Xbi...wX4WFoz" />
```

This value should be long, increment in an unpredictable way or contain a timestamp, and be generated with a static secret.

Each time the server serves the form, it should generate a new CSRF token and include it in the form. If a hacker tries to create a CSRF exploit by including the hidden field they see when they examine the form's HTML source, the exploit will fail because the server code will check and see that the increment value or timestamp in the attack form is incorrect.

Insecure Direct Object Reference

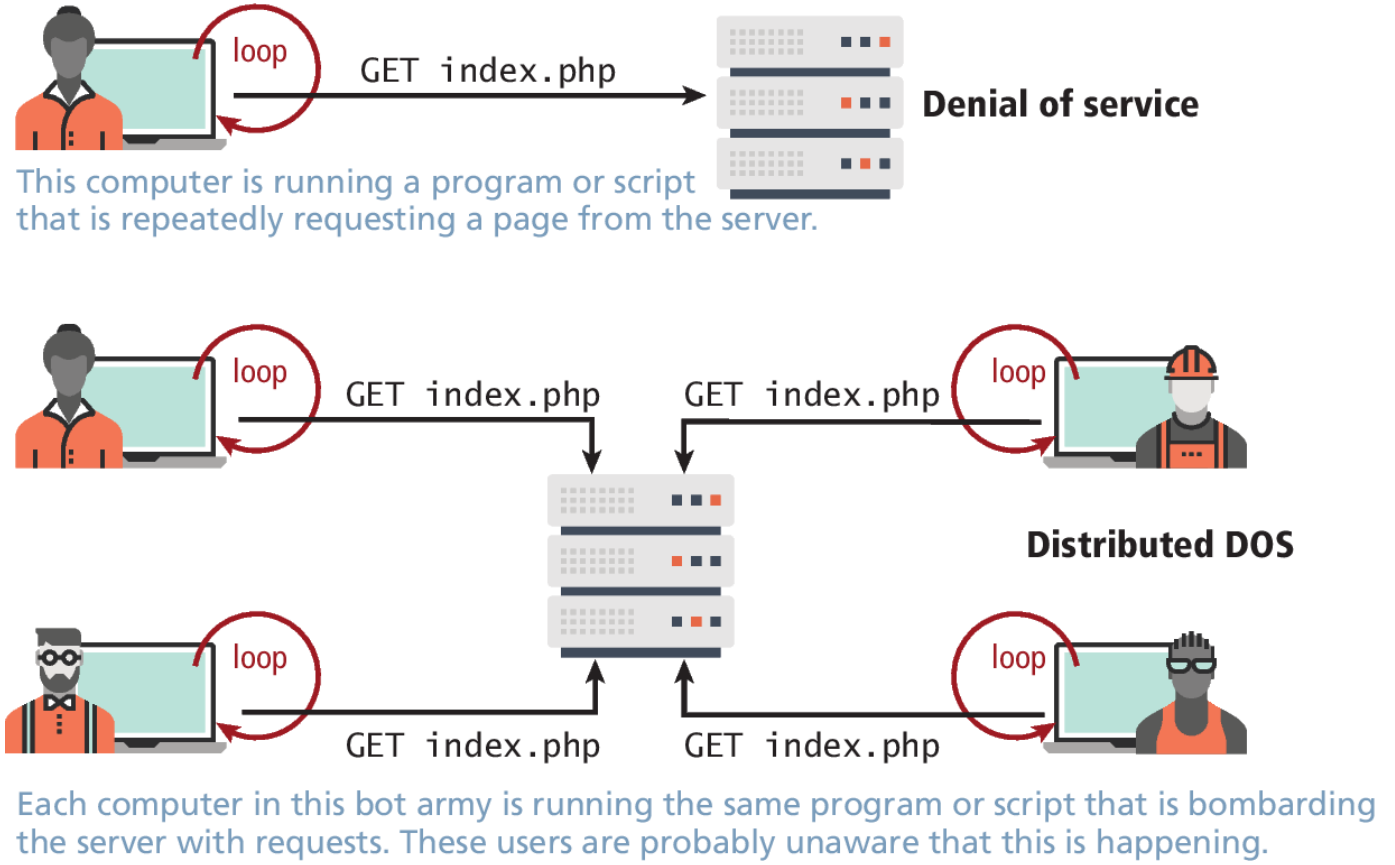
An **insecure direct object reference** is a fancy name for when some internal value or key of the application is exposed to the user, and attackers can then manipulate these internal keys to gain access to things they should not have access to.

For instance, if a user can determine that his or her uploaded photos are stored sequentially as **/images/99/1.jpg**, **/images/99/2 .jpg**, . . . , they might try to access images of other users by requesting **/images/ 101/1.jpg**

Denial of Service

Denial of service attacks (DoS attacks) are attacks that aim to overload a server with illegitimate requests in order to prevent the site from responding to legitimate ones.

Distributed DoS Attack (DDoS) distribute requests from multiple machines



Security Misconfiguration

The broad category of security misconfiguration captures the wide range of errors that can arise from an improperly configured server.

- Out-of-Date Software
- Open Mail Relays
- Virtual Open Mail Relay
- Arbitrary Program Execution

Key Terms

asymmetric cryptography
auditing
authentication
authentication factors
authentication policy
authorization
availability
bearer authentication
block ciphers
Certificate Authority
cipher
CIA triad
code review
confidentiality
Content Security Policy
cross-site request forgery (CSRF)
cross-site scripting (XSS)
cryptographic hash functions
decryption
denial of service attacks
digest
digital signature
domain-validated certificates
encryption
extended-validation certificates
form-based authentication
hash functions
high-availability
HTTP basic authentication
HTTP Token Authentication
Hypertext Transfer Protocol Secure (HTTPS)
information assurance
information security
insecure direct object reference
integrity
JWT (JSON Web Token)
key
legal policies
logging
man-in-the-middle attacks
multifactor authentication
open authorization (OAuth)
open mail relay
organization-validated certificates
pair programming
password policies
phishing scams
principle of least privilege
public key cryptography
rainbow table
reflected XSS
salting
secure by default
secure by design
Secure Sockets Layer
security testing
security theater
self-signed certificates
single-factor authentication
social engineering
stateless authentication
stored XSS
SQL injection
STRIDE
substitution cipher
symmetric ciphers
threat
Transport Layer Security (TLS)
unit testing
usage policy
vulnerabilities

Copyright